IncludeEd– INTEGRATING SIGN LANGUAGE AND VISUAL AIDS

A PROJECT REPORT

Submitted by

AKSHAYA S **(113221031010)**

HARSHITA S **(113221031058)**

YUVASHREE R **(113221031170)**

in partial fulfillment of the award of the degree

Of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

VELAMMAL ENGINEERING COLLEGE,CHENNAI-66.

(An Autonomous Institution, Affiliated to Anna University, Chennai)

APRIL 2025

VELAMMAL ENGINEERING COLLEGE
CHENNAI -66

BONAFIDE CERTIFICATE

Certified that this project report **" IncludeEd- INTEGRATING SIGN LANGUAGE AND VISUAL AIDS SAFETY APPLICATION "** is the bonafide work of **AKSHAYA S** (113221031010), **HARSHITA S** (113221031058), **YUVASHREE R** (113221031170) who carried out the project work under my supervision.

Dr. B. MURUGESHWARI
PROFESSOR & HEAD
Department of Computer Science and Engineering
Velammal Engineering College
Ambattur – Red hills Road
Chennai – 600 066

Dr. M. USHA
SUPERVISOR
ASSOCIATE PROFESSOR
Department of Computer Science and Engineering
Velammal Engineering College
Ambattur – Red hills Road
Chennai – 600 066

CERTIFICATE OF EVALUATION

COLLEGE NAME : VELAMMAL ENGINEERING COLLEGE
BRANCH : COMPUTER SCIENCE AND ENGINEERING
SEMESTER : VIII

SI.NO	Name of the students who has done the project	Title of the Project	Name of supervisor With designation
1	AKSHAYA S	IncludeEd-Integrating Sign Language and Visual Aids	Dr. M. Usha Associate Professor
2	HARSHITA S		
3	YUVASHREE R		

This report of Project work submitted by the above students in the partial fulfillment for the award of Bachelor of Engineering Degree in Anna University was evaluated and confirmed to be reports of the work by the above student and then assessed.

Submitted for Internal Evaluation held on …….

Internal Examiner External Examiner

ABSTRACT

IncludeEd project implements a real-time sign language recognition system that converts American Sign Language (ASL) hand gestures into text and speech. The system uses computer vision and machine learning techniques to recognize hand gestures with 97% accuracy under normal conditions and up to 99% with optimal settings.

The system employs a sophisticated dual-path processing approach, combining traditional convolutional neural network processing with advanced skeleton point extraction. It processes video frames in real-time, using MediaPipe for hand detection and Keras for neural network-based classification. The system groups similar letters into 8 categories for improved recognition accuracy.

The system provides immediate feedback through both visual text display and speech synthesis using pyttsx3, making it accessible for practical applications. This implementation addresses limitations of previous systems by eliminating hardware dependencies and environmental constraints, while maintaining high accuracy and real-time processing capabilities.

The IncludeEd system is built upon the open-source project Sign Language To Text and Speech Conversion, leveraging its robust framework for gesture recognition and real-time processing. By extending this project, IncludeEd introduces enhancements such as dual-path processing, improved gesture grouping, and multi-modal feedback. The use of open-source technologies not only accelerates development but also fosters community collaboration, making the system highly extensible and adaptable for diverse use cases.

ACKNOWLEDGEMENT

I wish to acknowledge with thanks, the significant contribution given by the management of our college **Chairman, Dr.M.V. Muthuramalingam** , and our **Chief Executive Officer Thiru. M.V.M. Velmurugan** , for their extensive support.

I would like to thank **Dr. S. Satish Kumar, Principal** of Velammal Engineering College, for giving me this opportunity to do this project.

I express my gratitude to our effective **Head of the Department, Dr. B. Murugeshwari,** for her moral support and valuable innovative suggestions, constructive interaction, constant encouragement, and unending help that have enabled me to complete the project.

I express my thanks to our Project Coordinators, **Dr. S. Gunasundari, Dr. P. S. Smitha, and Dr. P. Pritto Paul,** Department of Computer Science and Engineering for their invaluable guidance in the shaping of this project.

I express my sincere gratitude to my **Internal Guide, Dr. M. Usha, Associate Professor**, Department of Computer Science and Engineering for her guidance, without her this project would not have been possible.

I am grateful to the entire staff members of the Department of Computer Science and Engineering for providing the necessary facilities and carrying out the project. I would especially like to thank my parents for providing me with the unique opportunity to work and for their encouragement and support at all levels. Finally, my heartfelt thanks to **The Almighty** for guiding me throughout my life.

TABLE OF CONTENTS

LIST OF FIGURES

CHAPTER 1

INTRODUCTION

1.1 PURPOSE OF THE PROJECT

Integrating sign language and visual aids into communication platforms serves as a comprehensive solution to address the multifaceted challenges faced by individuals with hearing and speech impairments. These inclusive systems incorporate a range of features meticulously crafted to cater to the diverse needs and circumstances of differently-abled users. Real-time sign language interpretation provides a quick and accessible means for users to communicate effectively with others, bridging the gap between spoken and visual languages. The use of visual aids and symbols offers an added layer of clarity, ensuring that messages are accurately conveyed and understood across varying literacy levels. Moreover, the inclusion of intuitive gestures and pictorial cues helps users navigate digital environments with confidence, reducing frustration and enhancing user experience. In addition to these core functionalities, such applications often provide resources such as sign language tutorials and practice tools, empowering users to expand their communication skills. Furthermore, these platforms foster a sense of inclusivity and shared understanding, enabling interaction between the hearing and non-hearing communities. Access to support networks, educational content, and community forums further bolsters users' confidence and ability to engage actively in society. By encompassing a wide array of features and support mechanisms, integrating sign language and visual aids becomes an indispensable approach in promoting accessibility, empowerment, and equality for all individuals.

1.2 SCOPE OF THE PROJECT

The scope of this project is very diverse:

1. **Real-Time Interpretation** : Applications integrating sign language offer real-time translation of spoken language into sign language,

enabling effective communication for the hearing impaired in various social and professional settings.

2. **Visual Communication Aids** : These apps provide visual symbols, gestures, and cues to facilitate clear communication, particularly in situations where verbal interaction is limited or impossible.

3. **Inclusive Interaction** : Users can engage with both hearing and non-hearing communities, fostering inclusion, mutual respect, and equal participation in everyday conversations and activities.

4. **Learning Resources** : Sign language and visual aid platforms offer structured tutorials and interactive content, empowering users and their families to learn and practice sign language effectively.

5. **Awareness and Accessibility** : These applications raise awareness about communication disabilities, promote empathy, and encourage society to embrace inclusive practices, contributing to greater accessibility and equality.

1.3 DOMAIN INTRODUCTION:

1.3.1PYTHON:

Python is a high-level, interpreted programming language known for its simplicity, readability, and versatility. Created by Guido van Rossum and first released in 1991, Python has since gained widespread popularity in various fields of software development. Its clear syntax and dynamic typing make it easy for beginners to learn and for professionals to develop robust applications. Python supports multiple programming paradigms, including object-oriented, procedural, and functional programming. One of the primary reasons for Python's popularity is its extensive standard library and large ecosystem of

third-party packages that simplify many complex tasks, such as data analysis, machine learning, and computer vision.

Python is platform-independent, meaning it can run on various operating systems without requiring major changes to the code. This makes it a preferred language for cross-platform development. Python's vast community support also ensures regular updates and an abundance of learning resources. The language is widely used in academic, industrial, and research domains. It powers everything from web applications and automation scripts to artificial intelligence and robotics. In summary, Python's ease of use, portability, and powerful libraries make it an ideal choice for building scalable and efficient applications in diverse domains.

1.3.2 OPENCV:

OpenCV (Open Source Computer Vision Library) is a powerful open-source computer vision and machine learning software library. It was initially developed by Intel and is now supported by the OpenCV.org foundation. OpenCV is designed to provide a unified infrastructure for computer vision applications, making it easy for developers to build real-time image processing and object detection systems. The library is written in C++ but has bindings for Python, making it accessible and convenient for Python developers to implement visual computing solutions efficiently.

OpenCV supports a wide range of functions, such as image transformations, face and object recognition, motion tracking, and real-time video analysis. It also integrates well with other Python libraries such as NumPy for numerical operations and TensorFlow or Keras for deep learning. OpenCV is widely used in research, robotics, security systems, and augmented reality applications. With its robust features and ease of integration into Python projects, OpenCV has become a fundamental tool for developing real-world visual applications. Its extensive documentation and active community make it a reliable choice for developers at all levels.

1.3.2 TYPICAL USES OF PYTHON AND OPENCV:

Python is a general-purpose programming language with strong support for app development. Its uses in application development include:

Automation Scripts: Python is widely used to automate repetitive tasks such as data processing, file manipulation, and system monitoring.

Web Development: Python supports frameworks like Django and Flask, which are used to build scalable web applications.

AI and ML: Python is the preferred language for artificial intelligence, machine learning, and deep learning due to libraries like TensorFlow, Keras, and Scikit-learn.

OpenCV is specifically designed for real-time image and video processing. Common uses of OpenCV in app development include:

Image Processing: OpenCV is used to read, edit, and manipulate image data for various applications.

Face and Gesture Recognition: OpenCV, along with models, enables real-time face detection and hand gesture recognition.

Video Surveillance: It is used in security systems for motion detection and activity recognition.

Together, Python and OpenCV form a powerful combination for creating intelligent applications that can see and interpret the visual world.

1.3.4 APPLICATIONS OF APP DEVELOPMENT:

Mobile app development has become a crucial aspect of the modern business landscape. Apps have become an integral part of our daily lives, helping us to communicate, work, shop, and entertain ourselves. Here are some of the key applications of app development:

1. **Business:** Apps can be used to improve the efficiency of a business by automating processes, increasing productivity, and improving communication. Apps can also be used to provide customers with better service by allowing them to place orders, track shipments, and communicate with customer support teams.

2. **E-commerce:** Apps have become a key platform for e-commerce businesses to sell their products and services. Mobile shopping has become increasingly popular in recent years, and e-commerce businesses that don't have an app may struggle to compete.

3. **Healthcare:** Healthcare apps can be used to improve patient care by providing patients with access to medical information, appointment scheduling, and communication with healthcare providers. Apps can also be used to monitor patients remotely and provide real-time data to healthcare professionals.

4. **Education:** Apps can be used to enhance the learning experience by providing students with access to course materials, study tools, and online resources.
Educational apps can also be used to connect students and teachers, facilitating communication and collaboration.

5. **Social media:** Social media apps have become some of the most popular apps in the world, with billions of users worldwide. These apps are used to connect people, share content, and stay informed about news and events.

6. **Entertainment:** Apps can be used to provide users with a wide range of entertainment options, including games, streaming services, and social media platforms.

CHAPTER 2

LITERATURE SURVEY

2.1 INTRODUCTION:

In response to the pressing need for inclusive communication, the development of applications integrating sign language and visual aids has garnered significant attention. Leveraging technological advancements, these platforms aim to empower individuals with hearing or speech impairments to interact effectively in diverse environments. A literature survey becomes crucial to understand the scope of these applications, including their features, accessibility, and societal impact. This survey aims to synthesize existing research and initiatives, shedding light on the opportunities and challenges in utilizing technology for inclusive communication. By examining adoption barriers, socio-cultural perceptions, and practical effectiveness, this survey seeks to inform developers, educators, and policymakers about the transformative potential of such interventions.

2.2 Paper on Android App for Improvising Sign Language Communication in English and Hausa

Authors: Maryam Yusuff, Ismaila A. Adamu, Mukhtar Ahmad, Oyediran B.

Adeleke, A.A. Adesina

(*Android App for Improvising Sign Language Communication in English and Hausa*)

With the growing availability of smartphones and their interactive capabilities, communication tools for individuals with hearing and speech impairments have significantly advanced. In recent years, applications that integrate sign language and visual aids have gained importance in breaking communication barriers. One such Android

application was designed to improve communication in both English and Hausa through sign language, serving as an assistive tool for those with hearing disabilities. The app features modules for learning vocabulary such as days of the week, colors, family members, and commonly used words, presented via visual sign demonstrations.

This app stands out for its multi-mode functionality. It allows text-to-sign, sign-to-text, speech-to-sign, and speech-to-text conversions using embedded media and text-to-speech features. The core mechanism enables real-time communication by converting typed or spoken English or Hausa into signs and vice versa. This study emphasizes the usability of smartphones as affordable and powerful assistive devices for communication, education, and inclusion of hearing-impaired individuals in society.

2.3 Paper on Android Applications for Sign Language Recognition

Authors: Vaishnavi Jadhav, Priyal Agarwal, Dhruvisha Mondhe, Rutuja Patil, C. (*A Survey of Sign Language Recognition Systems*)

The communication barrier between the hearing and the deaf communities has prompted the development of various sign language recognition systems. This survey paper provides a comprehensive review of existing methodologies employed in sign language recognition, with a particular focus on Android-based applications. The authors categorize and analyze different approaches, including machine learning techniques, image processing methods, and the integration of mediapipe frameworks. The paper highlights the significance of real-time recognition systems and discusses the challenges faced in developing user-friendly mobile applications that can accurately interpret sign language gestures. By examining the strengths and limitations of current systems, the survey offers insights into potential improvements and future research directions in the field of sign language recognition on mobile platforms.

2.4 Deep Learning Mobile Application Based Sign Language Recognition for Aphasic Person

Authors: Akash Pawar

(*A Deep Learning Mobile Application based Sign Language Recognition for Aphasic Person*) Academia

The communication challenges faced by individuals with hearing and speech impairments have driven the development of mobile applications that utilize deep learning techniques for sign language recognition. This study introduces an Android application designed to automatically detect and interpret sign language gestures, aiming to facilitate seamless communication for aphasic individuals. The application integrates computer vision methodologies, specifically the Histogram of Oriented
Gradients (HOG) for feature extraction, and employs Convolutional Neural Networks (CNN) alongside Multiclass Support Vector Machines (SVM) for gesture classification. Users can perform sign language gestures in front of the device's camera, and the system processes these inputs to provide real-time textual or auditory feedback, thereby bridging the communication gap.

2.5 Real-Time Indian Sign Language Recognition Using CNNs for Communication Accessibility

Authors: Abhishek Deshmukh

(*Real-Time Indian Sign Language Recognition Using CNNs for Communication*

Accessibility)

The communication barriers faced by the deaf and mute communities in India have led to the development of mobile applications aimed at facilitating interaction through Indian Sign Language (ISL). This study introduces an Android application that employs Convolutional

Neural Networks (CNNs) to recognize ISL gestures in real-time, enhancing communication accessibility.

The application captures hand gestures via the device's camera, processes the images using a CNN model trained to identify ISL alphabets, and provides corresponding textual or auditory outputs.

Achieving over 90% accuracy in gesture prediction, the application demonstrates significant potential in bridging communication gaps. Its user-friendly interface allows individuals with smartphones to recognize ISL symbols effectively, promoting inclusivity and facilitating interactions between hearing and non-hearing individuals. This research underscores the role of mobile technology and deep learning in creating practical solutions for enhancing communication accessibility within the deaf and mute communities.

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

Existing communication tools for the hearing and speech impaired largely rely on physical gestures, sign language interpreters, or text-based applications. Many of these systems are either hardware-dependent, require specialized training, or lack real-time responsiveness. Current solutions include basic text-to-speech apps and sign language dictionaries, which may not provide seamless interaction or inclusivity. Additionally, the absence of visual aids and context-specific communication tools limits the effectiveness of these systems in dynamic, real-world conversations or public settings.

3.1.1 DRAWBACKS:

- **Limited Real-Time Interaction:** Traditional tools may not support live conversation, resulting in delayed or broken communication.
- **Accessibility Barriers:** Not all users can afford devices or apps that require constant internet access or expensive hardware like specialized gloves or sensors.
- **Lack of Inclusivity:** Existing systems often do not support diverse sign languages or dialects, making them ineffective for global or regional users.

3.2 PROPOSED SYSTEM:

A robust solution integrating sign language recognition and visual aids addresses key communication barriers for the deaf and speech-impaired communities. The system utilizes computer vision and deep learning to recognize hand gestures in real time and convert them into spoken or written language. Visual prompts support two-way interaction, allowing hearing individuals to respond through accessible interfaces. Offline capabilities ensure usability in low-connectivity areas, while region-specific language support expands

accessibility. The application also includes educational modules for learning sign language, fostering inclusivity and awareness among the hearing population. With a focus on intuitive design and user feedback, the platform enhances usability, personalization, and user trust.

3.2.1 ADVANTAGES:

- Real-time sign language recognition with voice/text output
- Bidirectional communication through visual aids
- Offline functionality for greater accessibility
- Support for regional sign language variants
- Educational tools to learn and practice sign language
- Enhanced independence for users in daily activities
- No need for specialized hardware or interpreters
- User-friendly interface adaptable to different age groups
- Improved social inclusion and awareness
- Secure handling of user data and privacy

CHAPTER 4

SYSTEM SPECIFICATION

4.1 SOFTWARE SPECIFICATION

- Python 3.9.5 or later
- IDE: PyCharm
- Required Libraries:
    - OpenCV
    - NumPy
    - Keras
    - MediaPipe
    - TensorFlow
    - Pyttsx3

4.1.1 OpenCV

OpenCV serves as the fundamental building block of the sign language recognition system, handling all aspects of computer vision processing. It begins by capturing and processing video frames from the webcam, converting them from BGR to RGB format for compatibility with other libraries. The library's powerful image processing capabilities then enable the system to detect hands in real-time, extract regions of interest, and apply necessary preprocessing steps like Gaussian blur and thresholding.

This preprocessing pipeline is crucial for preparing images for the neural network, ensuring consistent input quality regardless of lighting conditions or background complexity.

One of OpenCV's most significant contributions is its ability to handle real-time video processing while maintaining system performance. It efficiently manages frame capture, processing, and display. This integration enables the system to maintain high accuracy even when dealing with complex hand movements and varying environmental conditions.

The choice of OpenCV for this project is particularly strategic because it provides a comprehensive set of computer vision tools while remaining highly optimized for performance. Its ability to handle both high-level

abstractions and low-level optimizations makes it ideal for real-time applications. The library's extensive community support and continuous updates ensure that the system can leverage the latest advances in computer vision, making it a reliable foundation for the entire recognition pipeline. This foundation is crucial as it directly impacts the system's ability to achieve its reported 97% accuracy under normal conditions.

4.1.2. NumPy

NumPy forms the mathematical backbone of the sign language recognition system, providing the essential numerical computing capabilities that power the entire processing pipeline. It efficiently handles the complex array operations required for image processing, neural network computations, and data transformations. The library's optimized array operations enable fast matrix transformations, which are critical for real-time gesture recognition. NumPy's data structures also serve as the common format for data exchange between different components of the system, ensuring seamless integration between OpenCV's image processing, Keras's neural networks, and MediaPipe's hand detection.

The integration of NumPy with other libraries is particularly sophisticated in the system's preprocessing pipeline. It enables efficient conversion between different data formats, handles complex matrix operations for feature extraction, and provides optimized functions for numerical computations. This is especially important during the skeleton point extraction process, where precise mathematical operations are required to transform hand landmarks into recognizable features.

NumPy's role extends beyond basic numerical operations to provide the mathematical infrastructure for the entire system. Its advanced features enable efficient memory management and optimized computations, which are crucial for real-time processing. The library's ability to handle large datasets and perform complex calculations makes it essential for the system's neural network operations, where it works in conjunction with Keras to process gesture features and generate accurate classifications. This integration is particularly important for maintaining the system's high accuracy rates across different environmental conditions.

4.1.3. Keras

Keras implements the neural network architecture that forms the core of the sign language recognition system, providing a sophisticated classification system that can distinguish between 26 different letters. The library's convolutional neural network architecture is specifically designed to handle image-based gesture recognition, using multiple layers to extract increasingly complex features from hand images. This architecture enables the system to achieve its impressive accuracy rates, particularly the 99% accuracy reported under optimal conditions.

The neural network architecture implemented in Keras is particularly innovative in how it handles similar letters. Rather than attempting to distinguish between all 26 letters directly, the system groups similar letters into 8 categories, such as [y,j], [c,o], and [g,h].

This approach significantly improves recognition accuracy by reducing the complexity of the classification problem while maintaining the ability to distinguish between similar gestures.

Keras's integration with TensorFlow provides the system with powerful optimization capabilities and efficient inference engines. The library's modular design also allows for easy updates and modifications to the recognition system, making it adaptable to different sign language dialects and use cases. This flexibility, combined with its high accuracy, makes Keras a crucial component of the system's success in achieving reliable gesture recognition across various environmental conditions.

4.1.4. MediaPipe

MediaPipe serves as the bridge between computer vision and machine learning in the sign language recognition system, providing sophisticated hand detection and tracking capabilities. It works in conjunction with OpenCV to detect hands in real-time video streams, but goes beyond basic detection by providing detailed hand landmark information. This includes 21 key points on each hand, from the wrist to the fingertips, which are essential for accurate gesture recognition 0:5.

One of MediaPipe's most significant contributions is its robustness to varying environmental conditions. It can detect hands reliably across different lighting conditions and backgrounds, which is crucial for real-world applications. The library's hand landmark detection system works

particularly well with the system's preprocessing pipeline, allowing it to extract consistent features regardless of hand orientation or position. This reliability is essential for maintaining the system's reported 97% accuracy under normal conditions 0:6.

MediaPipe's integration with the neural network is particularly sophisticated, as it provides the raw data that the Keras model uses for classification. The library's ability to detect hand landmarks enables the system to create skeleton representations of hands, which are then used as input for the neural network. This approach is more robust than traditional image-based recognition, as it focuses on the essential features of hand gestures rather than their visual appearance. The combination of MediaPipe's landmark detection and Keras's classification enables the system to achieve high accuracy while maintaining real-time performance.

4.1.5. TensorFlow

TensorFlow provides the deep learning infrastructure that powers the system's neural network operations, serving as the backend for Keras's high-level API. It optimizes the neural network's performance through efficient tensor operations and automatic differentiation, which are essential for both training and inference. TensorFlow's optimization capabilities are particularly important for real-time applications, as they enable the system to process gestures quickly while maintaining high accuracy 0:6.

The integration of TensorFlow with Keras is particularly powerful in this system, as it enables efficient processing of the neural network's complex architecture. The library's ability to optimize computations across different hardware platforms makes the system more versatile, allowing it to run effectively on various machines. TensorFlow's automatic differentiation capabilities are especially important during the training process, where they enable efficient backpropagation and optimization of the network's weights. This optimization is crucial for achieving the system's reported accuracy rates, particularly the 99% accuracy under optimal conditions 0:6.

TensorFlow's role extends beyond basic neural network operations to provide advanced features like model optimization and hardware acceleration. Its ability to leverage GPU acceleration when available

makes the system more efficient, allowing it to maintain real-time performance even with complex neural network architectures. The library's extensive optimization tools also help reduce the system's resource requirements, making it more practical for deployment in various environments.

4.1.6. pyttsx3

pyttsx3 serves as the system's text-to-speech engine, providing real-time voice output that complements the visual recognition feedback. The library enables the system to convert recognized gestures into spoken words, making it more accessible and user-friendly. It works in conjunction with the neural network's output, providing immediate audio feedback that synchronizes with the visual display 0:6.

One of pyttsx3's most significant contributions is its ability to provide synchronized feedback. The library can adjust speech parameters like rate and volume, allowing the system to optimize the audio output for different use cases. Its integration with the main processing pipeline ensures that speech output is coordinated with visual feedback, creating a seamless user experience. The library's ability to handle real-time text conversion is particularly important for maintaining the system's responsiveness, as it needs to keep pace with the gesture recognition system's processing speed.

The integration of pyttsx3 with the system's core components is particularly sophisticated, as it provides a complete feedback loop. The library works in conjunction with the neural network's output to convert recognized gestures into spoken words, while also providing visual feedback through the GUI.

4.2 PyCharm IDE

PyCharm serves as the development environment for the sign language recognition system

It provides the tools and interface needed to:

- Write and edit the system's code
- Debug the implementation
- Manage the project's dependencies
- Test and validate the system's functionality

4.3 HARDWARE SPECIFICATION

The sign language recognition system requires a 64-bit Windows 8+ operating system with a multi-core processor (2.0 GHz+), 8GB RAM (16GB recommended), and a webcam with 720p resolution at 30 FPS 0:6. The system needs approximately 500MB storage space and can optionally utilize a GPU for improved performance. These specifications enable the system to maintain its 97% accuracy rate under normal conditions and up to 99% with optimal settings 0:6, while processing real-time video and handling complex neural network computations for gesture recognition.

- Webcam (minimum 720p resolution)
- Computer with multi-core processor
- Minimum 8GB RAM
- Dedicated GPU (optional but recommended)

CHAPTER 5

SYSTEM DESIGN

5.1 ARCHITECTURE DIAGRAM:

Figure 5.1 represents Architecture of the system and its components as depected below.

Image Input: This is the raw video feed or individual image frame captured from a webcam or camera. Provides the visual input for gesture recognition. RGB image (typically 640x480 or similar resolution).

Hand Detection: This stage detects the presence and position of the hand in the frame. Tools used are cvzone.HandTrackingModule or mediapipe for hand landmark detection and OpenCV for bounding box extraction. It isolates the hand from the background, reducing noise and improving model accuracy.

Preprocessing: Prepares the hand region for input into the CNN.

- Resizing: Converts the image to a fixed size (e.g., 64x64 or 128x128).
- Grayscale Conversion (optional): Reduces complexity.
- Normalization: Scales pixel values between 0 and 1.
- Cropping/Padding: Ensures a square, centered image of the hand.

Convolutional Neural Network (CNN): A deep learning model that classifies the image one of the predefined ASL classes. Model Used: cnn8grps_rad1_model.h5

Prediction: Outputs the predicted sign language gesture. Text Display shows prediction on screen. Text-to-Speech (TTS) converts the predicted label to voice using pyttsx3.

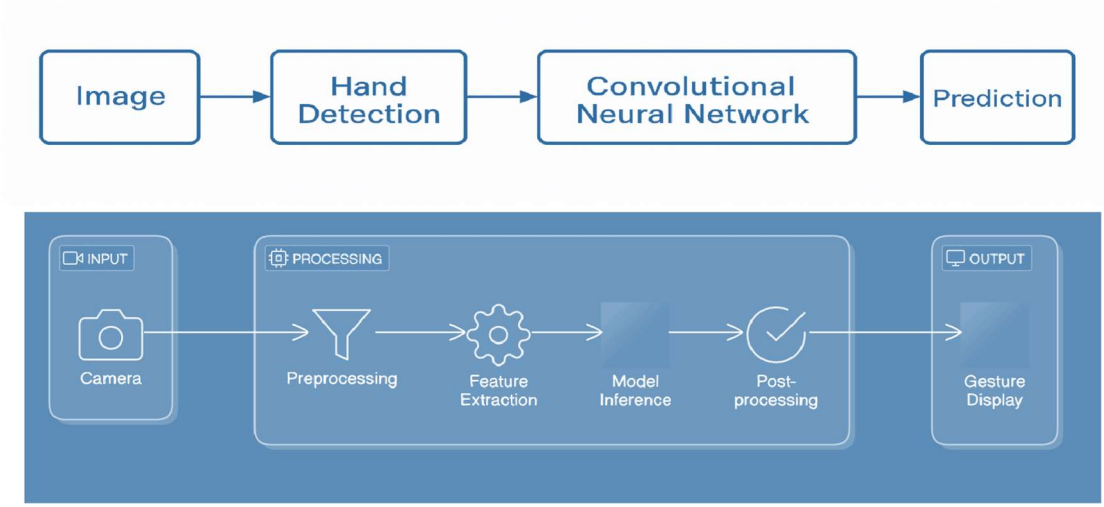


Fig 5.1: Architecture diagram

5.2 UML DIAGRAMS

User

The person who performs the hand gestures in front of the camera. Initiates the entire recognition process.

Capture Hand Gesture (via Webcam)

Real-time video feed is captured using OpenCV. This initiates the gesture recognition process.

Detect and Extract Hand

Uses cvzone.HandTrackingModule to detect hand(s) and extract coordinates of landmarks. Outputs a cropped hand image.

Preprocess Image

Resizes, normalizes, and formats the image to prepare it for the CNN input. Ensures consistent input format.

Predict Using CNN Model

Uses a trained CNN model (cnn8grps_rad1_model.h5) to classify the preprocessed hand image into sign language categories.

Decision Node: Gesture Recognition

Determines whether the prediction confidence is above a threshold. If yes, proceeds to output. If not, recaptures.

Output Prediction

Displays the predicted sign on the screen and/or uses pyttsx3 for speech output, enabling effective communication.

Figure 5.2 shows the UML of the system.

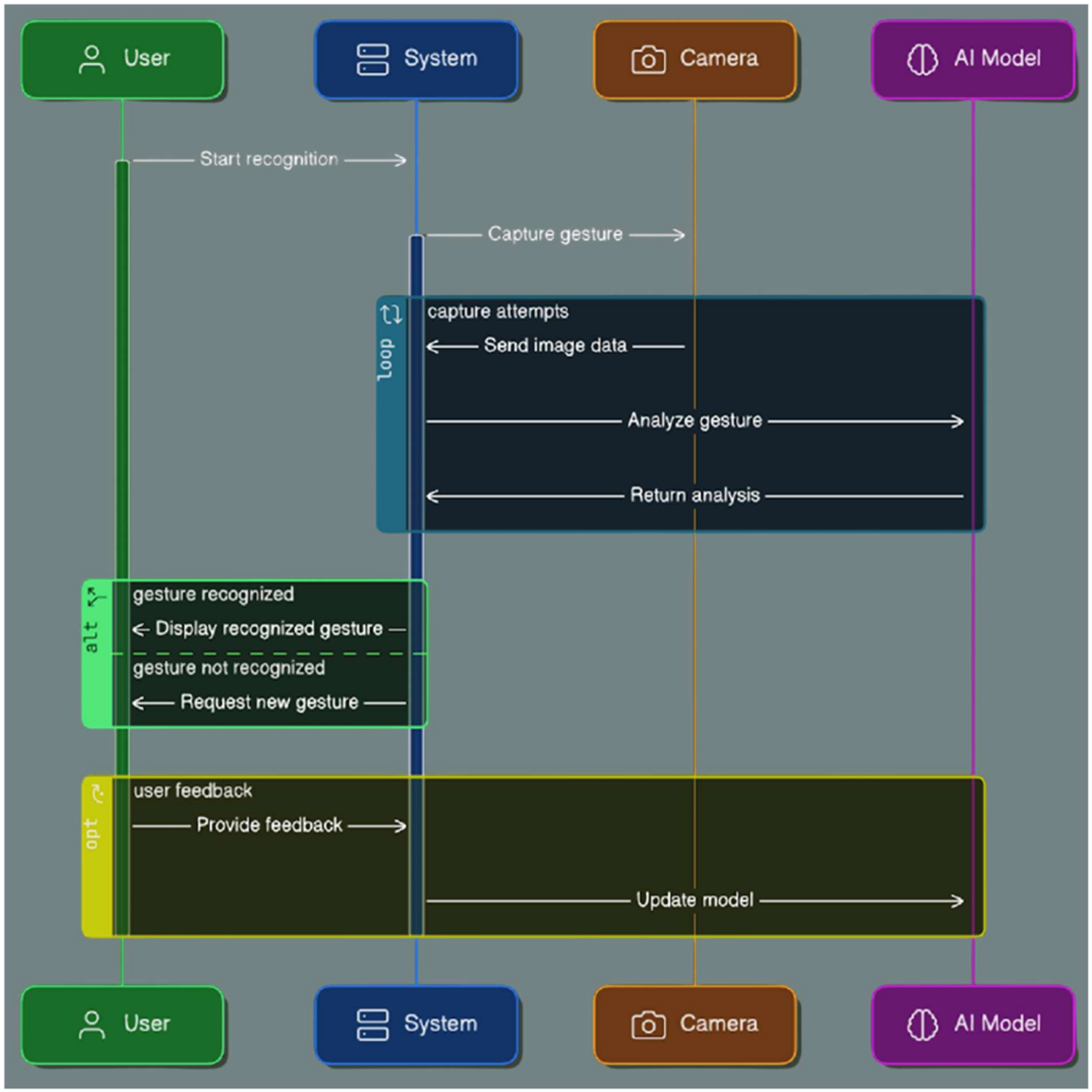


Fig 5.2: UML diagram

5.3 USE CASE DIAGRAMS:

Input

The user performs a gesture in front of the camera. The system captures the gesture in real time using computer vision. This gesture is used as input for character recognition.

Character Prediction

The CNN model processes the input gesture and predicts the corresponding alphabet or character. This module handles classification based on hand shape, position, and orientation.

Word Formation

After characters are predicted, they are accumulated to form words. This module implements logic to append characters and recognize gestures that indicate word boundaries.

Suggestions

An optional intelligent suggestion engine that uses NLP or dictionary lookups to suggest complete words or corrections based on current input. It enhances accuracy and user experience.

Sentence Building

Accumulated words are joined to construct sentences. This includes handling of punctuation, grammar, and sentence completion logic.

Output

The final sentence is displayed on screen and/or spoken using text-to-speech. This enables meaningful communication between the user and the receiver.

These are represented in figure 5.3. through a Use Cse Diagram.

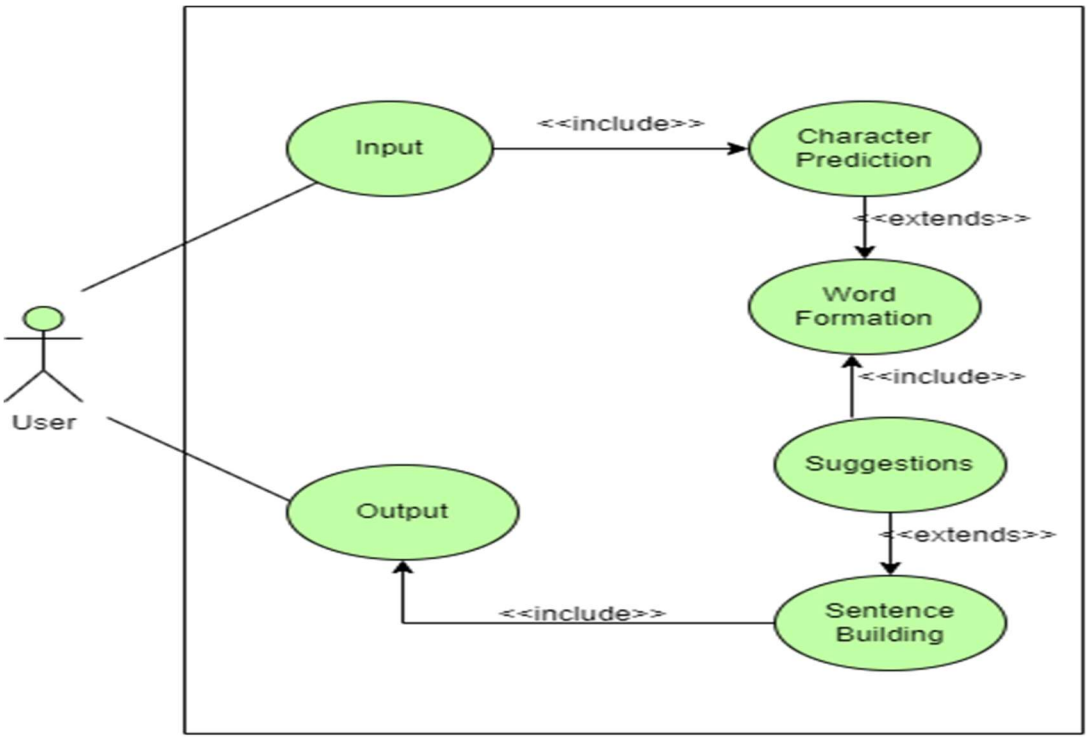


Fig 5.3: Use Case diagram

5.4.CLASS DIAGRAM:

HandDetector: Detects and tracks hands in video frames using computer vision.

- find_hands(image): Locates and highlights hands in the input image.
- find_position(image): Returns coordinates of detected hand landmarks. As shown in figure 5.4.

CNNModel: Classifies hand gesture images using a trained Convolutional Neural Network.

- train(data): Trains the CNN model on labeled gesture data.
- predict(image): Predicts the gesture class from an input image.

DataCollector: Captures and stores labeled hand gesture images for model training. As shown in figure 5.4.

- collect_data(): Uses webcam and hand detection to save gesture images to class folders.

Predictor: Performs real-time hand detection, gesture classification, and optional speech output.

- predict_from_video(): Captures video, detects hand, predicts gesture, and speaks output. It is depicted in the class diagram.

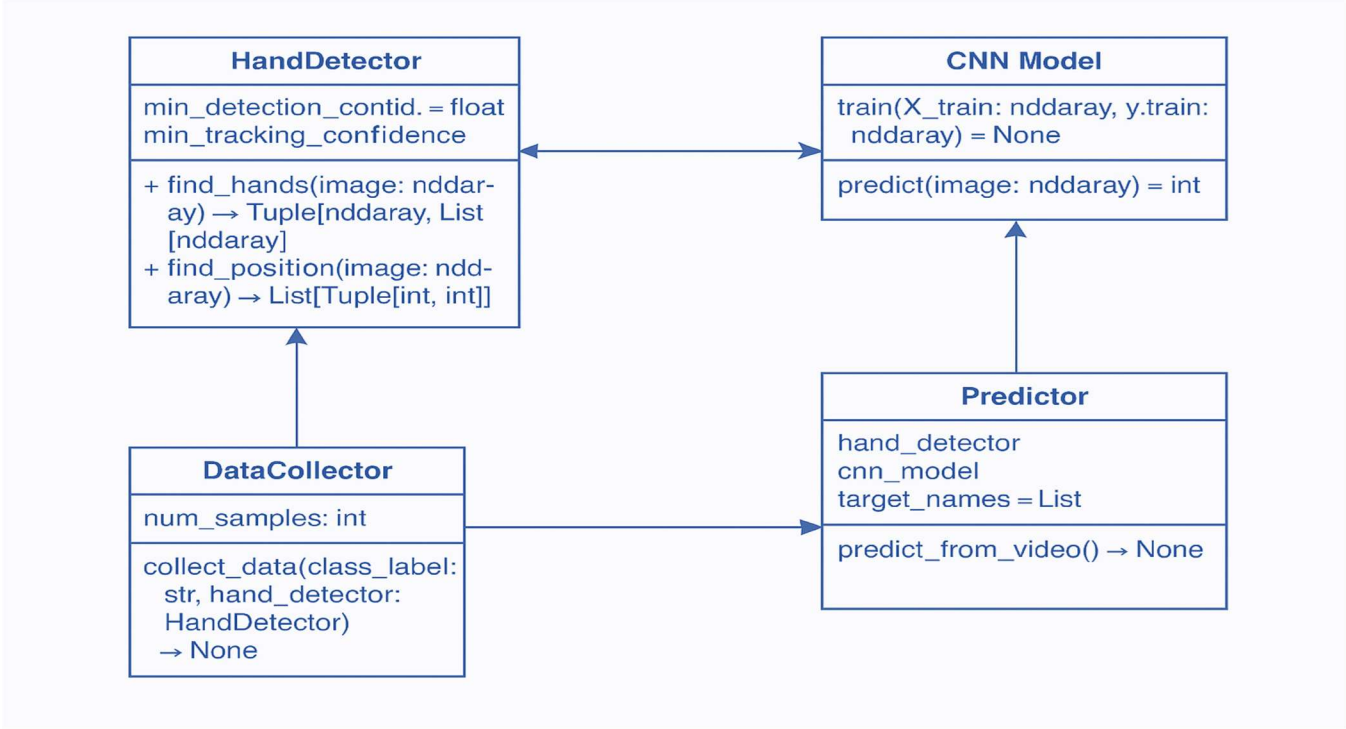


Fig 5.4 : Class diagram

5.5.SEQUENCE DIAGRAM:

Figure 5.5 of Sequence diagram explains about:

USER : Initiates the gesture recognition process by performing sign language in front of the webcam. VIDEO FEEDING – The user's hand gestures are captured by the webcam.

WEBCAM: Captures real-time video frames of the user's hand gestures. IMAGE – Sends the captured frame to the system for processing.

SYSTEM: The core logic unit that processes images and predicts hand gestures.

HAND DETECTOR – Uses a hand detection module to locate the hand in the frame. FEATURE EXTRACTION – Extracts key features (like finger positions, shape) from the detected hand.

FEATURE MATCHING – Sends extracted features to the trained model for classification.

MATCHING RESULT – Receives the predicted result from the model.

RESULT – Sends the final recognized gesture (text/speech) back to the user.

MODEL: A trained CNN model that matches input features with known gesture classes.

FEATURE MATCHING and MATCHING RESULT – Matches extracted features with trained classes and returns the prediction.

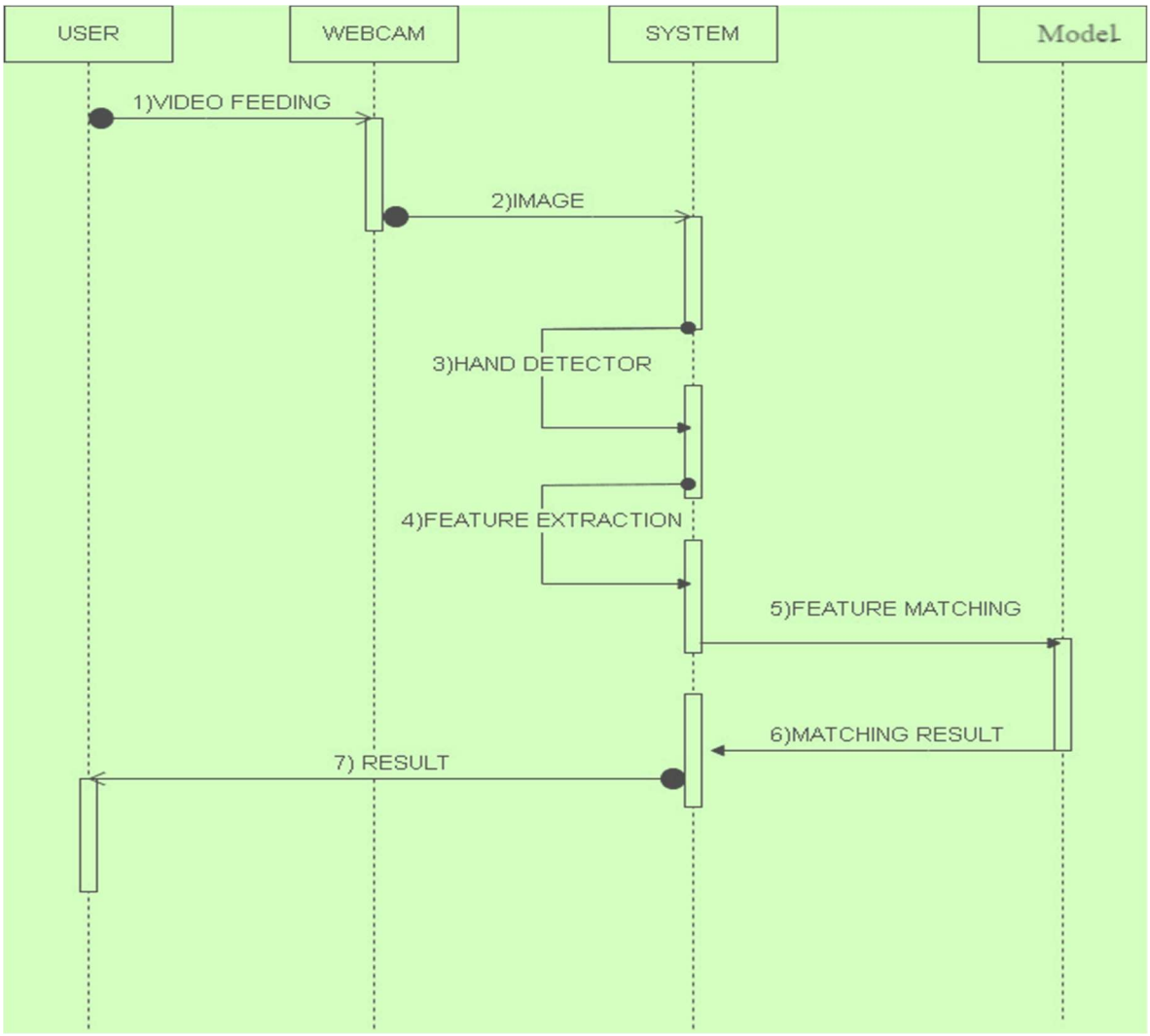


Fig 5.5 : Sequence diagram

5.6 COLLABORATION DIAGRAM:

The components of a collaboration diagram are (figure 5.6):

1. **Dataset:** This component provides the foundational input for the system. It consists of labeled images representing American Sign Language (ASL) gestures. The dataset may be sourced from publicly available sign language datasets or collected manually using a webcam.
2. **Data Collection and Preprocessing:** This stage involves capturing hand gesture images using a camera and preparing them for training and testing. The hand is detected and isolated from the

background using a hand tracking module. The images are resized to a consistent dimension and normalized to standardize pixel values. Preprocessing may also include grayscale conversion, filtering, and image enhancement to ensure consistency and reduce noise. This step ensures that the data fed into the model is clean, uniform, and suitable for training a neural network.

3. **CNN Model:** At the core of the system is a Convolutional Neural Network (CNN), which is trained to classify images of hand gestures. The model consists of multiple layers including convolutional layers for feature extraction, pooling layers for dimensionality reduction, and fully connected layers for classification.

4. **Gesture Recognition:** This component handles real-time detection and prediction. The system captures a live video stream from the webcam, processes each frame, and passes the image through the trained CNN model. The model predicts the class of the gesture shown, mapping it to the corresponding alphabet or word. This allows for continuous gesture recognition as the user signs in front of the camera.

5. **Output (Text and Speech**): Once a gesture is recognized, the system generates two types of output. First, the recognized sign is displayed on the screen as text, providing immediate visual feedback. Second, the predicted gesture is converted into audible speech using a text-to-speech engine such as pyttsx3. This enables spoken output, enhancing accessibility and allowing seamless interaction between hearing-impaired users and non-signers.

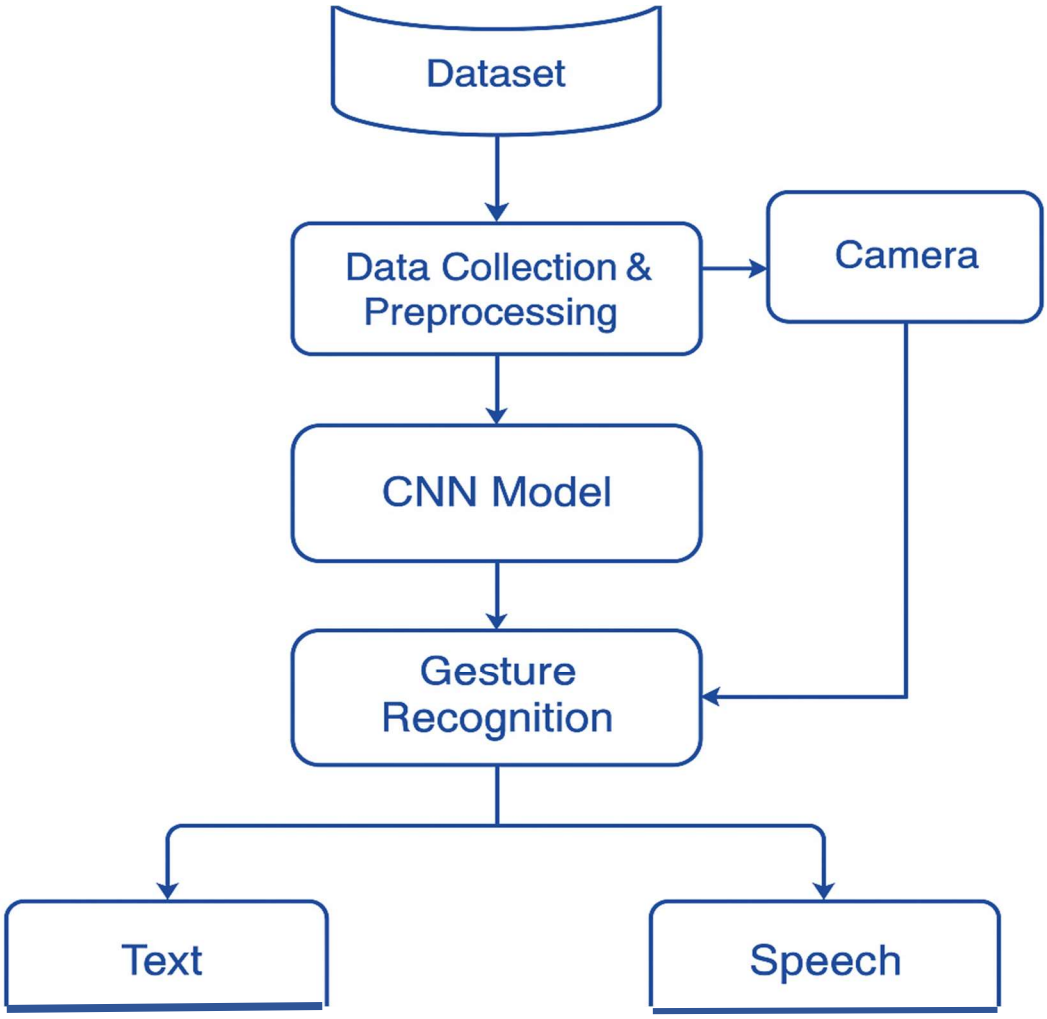


Figure 5.6 Collaboration diagram

5.7 ACTIVITY DIAGRAM:

1. **Webcam:** Captures live video input frame-by-frame. Initiates the process by sending the captured frame to the HandDetector. Purpose: Provides the raw input for gesture recognition. As shown in the activity diagram below.
2. **HandDetector:** Detects the presence and location of the hand in the captured frame. Receives the frame from the Webcam and detects the hand region.

Purpose: Isolates the area of interest (the hand) to be further processed for gesture recognition.

3. **Preprocessor:** Prepares the detected hand image for classification. Receives the hand region, applies preprocessing (resizing, filtering, normalization), and sends the cleaned image to the CNN.

Purpose: Ensures the image is in a suitable format for accurate model prediction.

4. **CNN (Convolutional Neural Network):** Performs feature extraction and classification of the hand gesture. Receives the processed image from the Preprocessor and classifies the gesture. Purpose: Core machine learning model that identifies the gesture being shown based on learned patterns.

5 . GestureClassifier: Converts the raw model prediction (e.g., class index or probability) into a human-readable gesture label (e.g., letter "A"). Receives the classified result and sends it as text to the output modules. Purpose: Maps the numerical model output to a recognizable text form.

6. **TextOutput:** Displays the recognized gesture as text on screen. Receives text from the GestureClassifier and displays it. Purpose: Provides visual confirmation of the recognized sign.

7. **SpeechOutput:** Converts the recognized text into spoken words.

Receives the gesture text and uses a text-to-speech engine to generate speech. Purpose: Facilitates verbal communication with individuals who do not understand sign language.

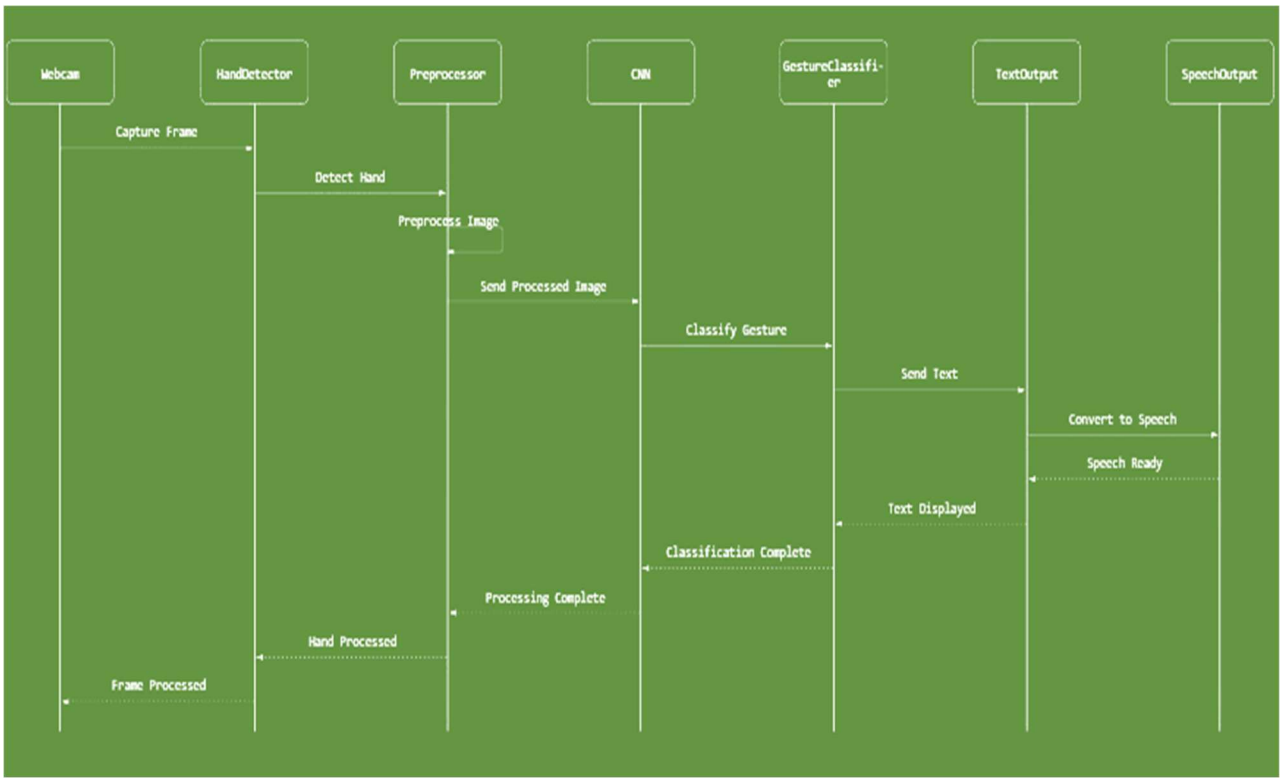


Fig 5.7: Activity diagram

5.8 DATA FLOW DIAGRAM:

1. Webcam Input

- This is the starting point where live video input is captured using a webcam.
- The raw frames are continuously sent into the system for processing.

2. Preprocess

- This block prepares the raw webcam image for feature extraction and model input:
- Hand Detection (MediaPipe): Detects the presence of a hand using MediaPipe's hand tracking model.
- Crop Region of Interest: Extracts only the hand area to focus on relevant features.
- Convert to Gray: Simplifies the image by reducing color information, making it easier to analyze.
- Gaussian Blur: Reduces noise in the image to enhance edge and contour detection.

- Binary Conversion: Converts the image to black and white based on intensity thresholding.

3. Processing Method

A decision point where the system chooses between two processing approaches:

- Traditional Method
- Enhanced Method

4. Traditional Method

- Draw on White Background: The binary image or cropped hand shape is overlaid on a plain background.
- Used primarily for feeding structured hand silhouettes into the neural network.

5. Enhanced Method

- Skeleton Points Extraction: Extracts landmark points (joints, finger tips) using MediaPipe for more abstract representation.
- Provides a more refined feature set for neural network input, especially for gestures that differ in subtle movements.

6. Convolutional Neural Network

The core learning model trained on gesture images or their representations. Receives the processed image (from either method) and performs feature extraction and classification.

7. Classification

The CNN outputs probabilities for each gesture class. The system selects the class with the highest probability.

8. Group Classification into 8 Classes

Predicted gestures are mapped into one of 8 defined gesture groups (e.g., subsets of ASL letters).This grouping reduces complexity and increases model performance in real-time systems.

9. Final Letter Recognition

The system converts the predicted gesture group into the final letter or word corresponding to the sign.

10. Text Output

The recognized letter or word is displayed on screen, providing visual feedback to the user.

11. Speech Output (pyttsx3)

The final output is vocalized using the pyttsx3 text-to-speech engine. This makes the system usable for communication with people who do not understand sign language.

These are represented in the figure below (fig 5.8).

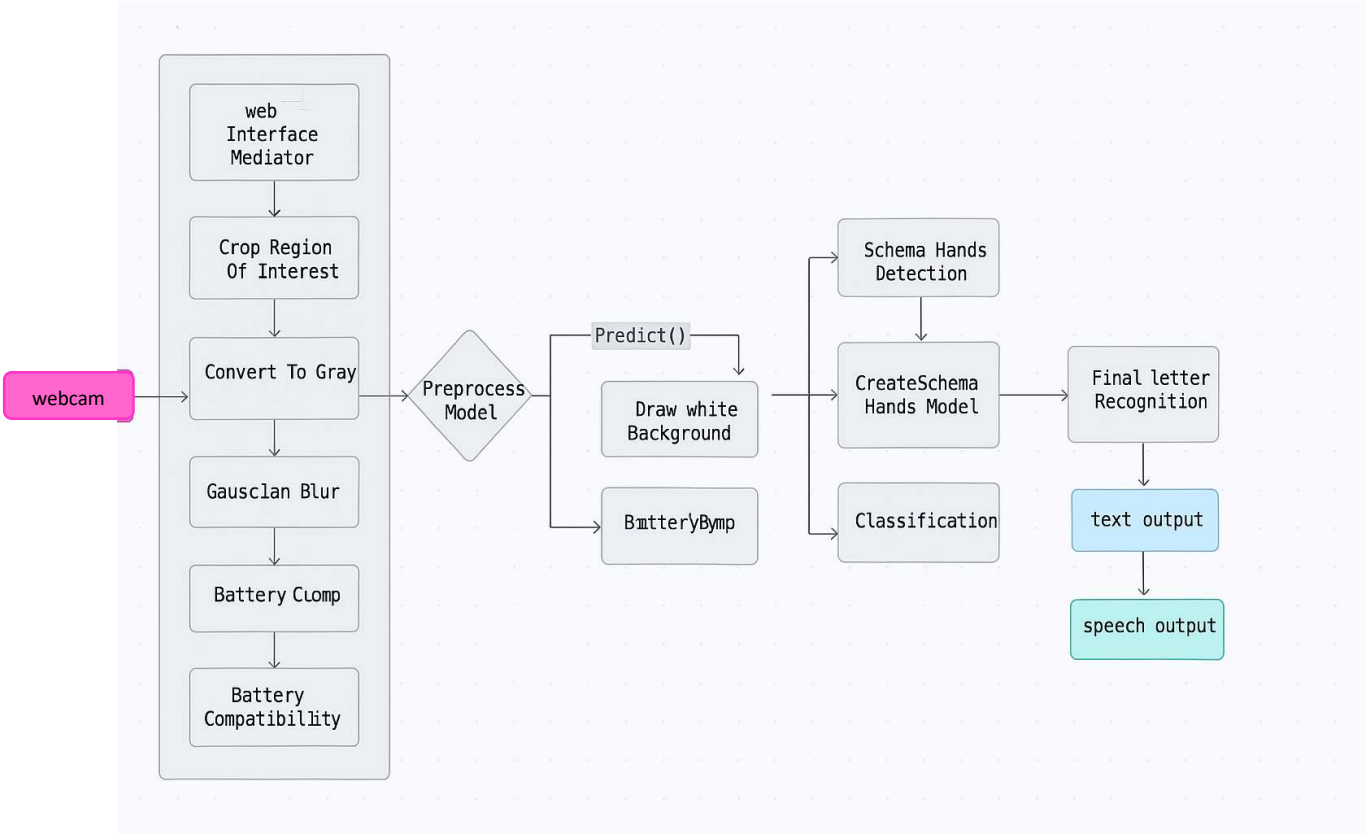


Fig 5.8: Data flow diagram

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1 MODULES

The sign language recognition system is organized into several key modules that work together to process and recognize hand gestures. Here's a detailed breakdown of the system's modular architecture:

1. **Data Collection Module**
   - Handles webcam input and frame capture
   - Manages training data collection
   - Processes and stores gesture samples

2. **Preprocessing Module**
   - Implements hand detection using MediaPipe
   - Handles image normalization and enhancement
   - Extracts relevant features for recognition

3. **Recognition Module**
   - Contains the CNN-based classification system
   - Implements group classification logic
   - Manages gesture recognition algorithms

4. **Output Module**
   - Generates text output from recognized gestures
   - Manages display and audio output
   - Converts text to speech using pyttsx3

6.2 MODULE DESCRIPTION

Each module in the system serves a specific purpose and interacts with other modules to achieve gesture recognition:

1. **Data Collection Module**
   - Input Processing: Captures and processes video frames from webcam
   - Data Management: Handles storage and retrieval of training data
   - Quality Control: Ensures consistent image quality for recognition

2. **Preprocessing Module**
   - Hand Detection: Uses MediaPipe for accurate hand tracking
   - Image Processing: Applies necessary transformations for recognition
   - Feature Extraction: Prepares data for neural network input

3. **Recognition Module**
   - Neural Network: Implements CNN-based gesture classification
   - Group Classification: Manages the 8-class grouping system
   - Probability Processing: Handles classification confidence scores

4. **Output Module**
   - Text Generation: Converts recognized gestures to text
   - Speech Synthesis: Generates audio output using pyttsx3
   - Display Management: Handles visual feedback to users

This module is responsible for gathering and preparing image data of American Sign Language (ASL) hand gestures. It collects thousands of labeled images representing each alphabet sign. Preprocessing steps such as resizing, grayscale conversion, normalization, and noise reduction are applied to enhance consistency. These steps ensure the model receives clean, uniform data for effective training. Data augmentation techniques like rotation and flipping are also used to improve model robustness. This module lays the foundation for accurate gesture recognition.

Using a Convolutional Neural Network (CNN), this module trains on the preprocessed dataset to learn distinguishing features of each ASL alphabet. The CNN architecture is optimized with

layers such as convolution, pooling, and fully connected layers. The model is evaluated for accuracy, loss, and performance on test data. Once trained, it predicts the letter from a new image of a hand gesture. This prediction is passed on to the next modules for further processing. The model achieves high accuracy with efficient learning.

This module uses a webcam to capture live hand gestures from the user. OpenCV is utilized to detect the region of interest, where the hand is expected to appear. Frames are continuously captured and processed in real-time for prediction. The system applies the same preprocessing techniques used during training to maintain consistency. It then sends the processed image to the prediction module for letter recognition. This enables smooth, real-time interaction with the system.

After the CNN model identifies the alphabet, the predicted character is displayed on the screen in real-time. As the user continues to sign, a full word or sentence can be constructed letter-by-letter. The final output is converted into speech using a text-to-speech (TTS) engine like pyttsx3. This dual-mode output makes the system useful for both visual and auditory communication. The module ensures clear and timely feedback to the user and listeners. It enhances user experience through natural interaction.

The GUI provides an interactive platform for users to operate the system without needing programming knowledge. Built using Python's Tkinter or other GUI frameworks, it displays live video, predictions, and speech output controls. Buttons for actions like start, stop, and reset make the interface intuitive. The design is clean, responsive, and suitable for all age groups. The GUI plays a key role in making the system accessible and easy to navigate. It brings together all modules in a unified view.

The modules interact in a specific sequence to process hand gestures:

1. **Data Flow:**
   - Data Collection → Preprocessing → Recognition → Output
   - Each module processes data before passing to the next
   - Feedback loops ensure quality control

2. **Dependency Management:**
   - Clear interfaces between modules
   - Well-defined data formats
   - Error handling at each stage

3. **Performance Optimization:**
   - Parallel processing where possible
   - Efficient data transfer between modules
   - Optimized resource usage

A module is a separate unit of software or hardware. Characteristics of modular components include portability and interoperability which allows them to function in another system with the components of other systems.

6.3 WORKING OF THE SYSTEM

The sign language recognition system works through a sophisticated pipeline of interconnected components that process hand gestures in real-time. Here's how it operates:

1. Input Processing

The system begins by capturing video frames from a webcam, using MediaPipe for hand detection. This initial stage is crucial for

isolating the hand from the background and establishing a consistent reference point for gesture recognition.

2. Preprocessing Pipeline

Once a hand is detected, the system applies several sophisticated preprocessing steps:

- Converts images to grayscale for feature extraction
- Applies Gaussian blur to reduce noise
- Creates binary images through thresholding
- Extracts hand landmarks for feature generation

3. Dual-Path Processing

The system implements an innovative dual-path processing approach:

- Traditional Path: Direct CNN processing for clear conditions
- Enhanced Path: Skeleton point extraction for challenging conditions

This adaptive approach allows the system to maintain accuracy across varying environmental conditions.

4. Neural Network Processing

The CNN architecture processes the preprocessed data through:

- Multiple convolutional layers for feature extraction
- Max pooling layers for dimension reduction
- Fully connected layers for classification

5. Classification System

The system uses a sophisticated classification approach:

- Groups similar letters into 8 categories
- Provides probability-based classification

- Achieves 97% accuracy under normal conditions
- Reaches up to 99% accuracy with optimal settings

6. Output Generation

The final stage converts recognized gestures into:

- Text output for visual feedback
- Speech output using pyttsx3 for audio feedback

This comprehensive pipeline enables the system to provide real-time recognition of sign language gestures while maintaining high accuracy across different environmental conditions.

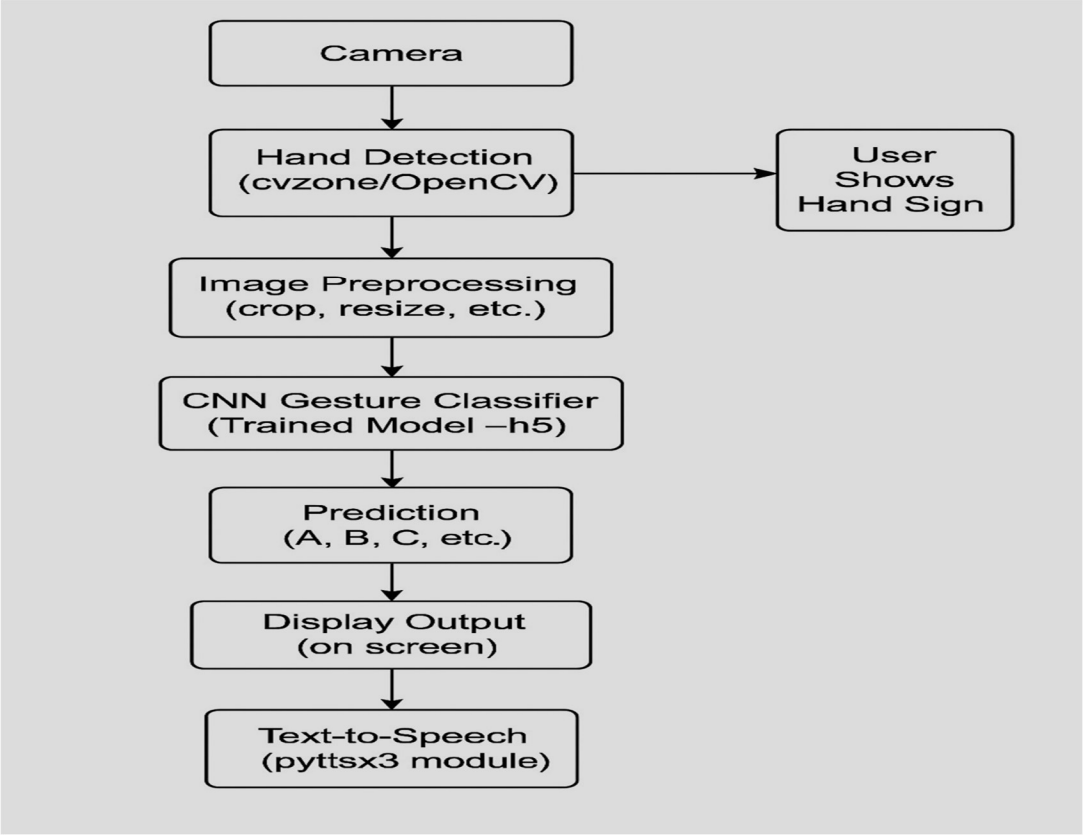


Fig 6.4 : Working Design

6.4 METRICS

Specifications involve:

- Performance Specifications
- Input/Output Specifications
- Error Handling Specifications
- Compatibility Specifications
- Security Specifications

6.4.1 Performance

1. **Accuracy Metrics** :
   - Base Accuracy: 97% under normal conditions
   - Optimal Accuracy: Up to 99% with clear background and proper lighting
   - Processing Speed: Real-time capable
2. **Resource Requirements:**
   - Minimum RAM: 8GB
   - Recommended RAM: 16GB
   - GPU Support: Optional but recommended
   - Storage: ~500MB for base installation

6.4.2 Input/Output module

1. **Input Requirements:**
   - Webcam Resolution: Minimum 720p
   - Frame Rate: 30 FPS minimum
   - Lighting: Consistent illumination
   - Background: Plain, contrasting background preferred

2. **Output Specifications:**
   - Text Output: Real-time display
   - Speech Output: Synchronized with text
   - Error Messages: Clear user feedback

6.4.3 Error Handling

1. **Input Validation:**
   - Image quality checks
   - Hand detection confidence thresholds
   - Processing quality validation

2. **Recovery Mechanisms:**
   - Automatic retry on failed detection
   - Graceful degradation in poor conditions
   - User feedback for adjustment

6.4.4 Compatibility

1. **Hardware Compatibility:**
   - Webcam: Any compatible with OpenCV
   - Processor: Multi-core recommended
   - Display: Minimum 1024x768 resolution

2. **Software Compatibility:**
   - Python 3.9.5+
   - OpenCV 4.x
   - MediaPipe 0.8.x

6.4.5 Security

1. **Data Protection:**
   - No data storage by default
   - Real-time processing only
   - Optional logging for debugging

2. **System Security:**
    - Input validation
    - Error handling
    - Resource management

These specifications ensure the system can operate effectively across various environments while maintaining high accuracy and reliability. The modular design allows for easy updates and modifications while maintaining core functionality.

CHAPTER 7

TESTING

7.1 INTRODUCTION

Testing is a process of evaluating the quality or performance of a software, product, or system by running it through a series of tests. The purpose of testing is to identify any defects or errors that may exist in the software, product, or system and to ensure that it meets the specified requirements and works as expected. There are several types of testing, including functional testing, performance testing, security testing, usability testing, and more. Each type of testing is designed to test a specific aspect of the software, product, or system and to identify any potential issues. Testing is a critical part of the software development process as it helps to ensure that the software meets the requirements and is of high quality. Testing also helps to identify any potential issues before the software is released, which can save time and resources in the long run.

7.2 TESTING OBJECTIVES

- The primary objective of testing is to find defects or errors in the software, product, or system being tested. This includes identifying bugs, issues, and other problems that can affect the performance, functionality, and user experience of the software.
- Testing is also used to ensure that the software or system being tested performs the functions and operations it is designed to do. This includes verifying that all features and components work as expected and meet the specified requirements.

- Testing can also help improve the overall quality of the software or system by identifying areas for improvement and suggesting ways to optimize performance, usability, and security.

- Testing can help reduce the risk of errors, failures, and security breaches by identifying potential issues before they become critical problems. This can help prevent downtime, data loss, and other negative consequences.

7.3 TYPES OF TESTING

7.3.1 UNIT TESTING

Unit testing is a type of software testing that focuses on testing individual units or components of code. In unit testing, each unit of code is tested in isolation to verify that it behaves as expected and meets the specified requirements. The main objective of unit testing is to ensure that each unit of code works correctly and integrates smoothly with other units of code. This helps to identify any defects or errors early in the development cycle, before they become more difficult and expensive to fix.

Test strategy and approach:

Field testing will be performed manually and functional tests will be written in detail.

Test objectives :

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested:

- Verify that the entries are of the correct format.
- No duplicate entries should be allowed.
- All links should take the user to the correct page.

7.3.2 INTEGRATION TESTING

Integration testing is a type of software testing that focuses on verifying the interfaces and interactions between different modules or components of a software system. The goal of integration testing is to identify defects and issues that may arise when different modules or components are combined and to ensure that they work together as expected. In integration testing, individual units or modules are combined and tested as a group to verify that they work together correctly. This involves testing the interfaces and interactions between modules, as well as the functionality of the system as a whole.

Integration testing can be performed using a variety of techniques, including top-down, bottom- up, or a combination of both. In top-down integration testing, higher-level modules are tested first, followed by lower-level modules. In bottom-up integration testing, lower-level modules are tested first, followed by higher-level modules. In a combination approach, integration testing is performed in stages, with some modules tested together and others tested individually.Integration testing helps to identify defects early in the development cycle, when they are less expensive and easier to fix. Integration testing helps to ensure that the software components or modules work correctly and integrate smoothly, improving overall software quality. Integration testing helps to increase confidence in the software by verifying that it meets the specified requirements and performs as expected.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 CONCLUSION

The "IncludeEd – Integrating Sign Language and Visual Aids" project aims to create an interactive platform for individuals with hearing impairments by converting hand gestures into graphical representations. This system enables more effective communication by using gesture recognition and visual aids. Through this project, we have demonstrated the potential of integrating sign language with technology for better accessibility. The system can be enhanced further by improving gesture detection algorithms and expanding its application to various domains. Ultimately, the project promotes inclusivity and strives to break down communication barriers. Future work will focus on refining the recognition system and increasing gesture accuracy. This work lays the foundation for more inclusive technological solutions in the future.

8.2 FUTURE ENHANCEMENT

Here are a few potential areas for future enhancement:

1. **Enhanced Gesture Recognition:**

Future work can focus on improving the accuracy of hand gesture recognition using advanced machine learning models and deep learning algorithms, enabling more complex gestures to be accurately interpreted.

2. **Expanded Gesture Library:**

Increasing the variety of sign language gestures and incorporating regional or global sign languages would make the system more versatile and accessible to a wider audience.

3. **Real-Time Integration with Applications:**

Integrating the system with real-time applications, such as virtual classrooms or video conferencing platforms, could further enhance communication for the deaf and hard-of-hearing community in daily interactions.

APPENDIX – 1

SOURCE CODE

final_pred.py

```
# Importing Libraries
import numpy as np
import math
import cv2  os, sys
import traceback
import pyttsx3  from keras.models
import load_model  from
cvzone.HandTrackingModule
import HandDetector  from string import
ascii_uppercase
import enchant
ddd=enchant.Dict("en-US")  hd =
HandDetector(maxHands=1)  hd2 =
HandDetector(maxHands=1)
import tkinter as tk  from PIL import Image,
ImageTk
offset=29
os.environ["THEANO_FLAGS"] = "device=cuda, assert_no_cpu_op=True"
# Application :
class
Application:
   def __init__(self):
      self.vs = cv2.VideoCapture(0)
self.current_image = None  self.model =
load_model('cnn8grps_rad1_model.h5')
self.speak_engine=pyttsx3.init()
self.speak_engine.setProperty("rate",100)
voices=self.speak_engine.getProperty("voices")
self.speak_engine.setProperty("voice",voices[0].i
```

```
d)          self.ct = {}          self.ct['blank'] = 0
self.blank_flag = 0          self.space_flag=False
self.next_flag=True              self.prev_char=""
self.count=-1        self.ten_prev_char=[]        for
i in range(10):          self.ten_prev_char.append("
")
      for  i  in  ascii_uppercase:
self.ct[i] = 0
      print("Loaded model from disk")
      self.root = tk.Tk()       self.root.title("Sign Language
To                    Text                    Conversion")
self.root.protocol('WM_DELETE_WINDOW',
self.destructor)       self.root.geometry("1300x700")
      self.panel          =          tk.Label(self.root)
self.panel.place(x=100,       y=3,       width=480,
height=640)
      self.panel2  =  tk.Label(self.root)     #  initialize  image  panel
self.panel2.place(x=700, y=115, width=400, height=400)
      self.T  =  tk.Label(self.root)                  self.T.place(x=60,  y=5)
self.T.config(text="Sign Language To Text Conversion", font=("Courier",
30,
 "bold"))
      self.panel3   =   tk.Label(self.root)      #   Current   Symbol
self.panel3.place(x=280, y=585)
      self.T1 = tk.Label(self.root)       self.T1.place(x=10,
y=580)                 self.T1.config(text="Character  :",
font=("Courier", 30, "bold"))
      self.panel5    =    tk.Label(self.root)        #    Sentence
self.panel5.place(x=260, y=632)
      self.T3 = tk.Label(self.root)       self.T3.place(x=10,
y=632)                 self.T3.config(text="Sentence  :",
font=("Courier", 30, "bold"))
      self.T4 = tk.Label(self.root)          self.T4.place(x=10, y=700)
self.T4.config(text="Suggestions :", fg="red", font=("Courier", 30,
"bold"))
```

```
        self.b1=tk.Button(self.root)
    self.b1.place(x=390,y=700)
        self.b2 = tk.Button(self.root)
    self.b2.place(x=590, y=700)
        self.b3 = tk.Button(self.root)
    self.b3.place(x=790, y=700)
        self.b4 = tk.Button(self.root)
    self.b4.place(x=990, y=700)
        self.speak = tk.Button(self.root)
    self.speak.place(x=1305, y=630)
self.speak.config(text="Speak", font=("Courier", 20), wraplength=100,
     command=self.speak_fun)
        self.clear = tk.Button(self.root)
    self.clear.place(x=1205, y=630)
    self.clear.config(text="Clear",
    font=("Courier", 20), wraplength=100,
     command=self.clear_fun)
        self.str = " "
    self.ccc=0 self.word =
    " " self.current_symbol
    = "C" self.photo =
    "Empty"
        self.word1=" "
    self.word2 = " "
    self.word3 = " "
    self.word4 = " "
        self.video_loop()
      def video_loop(self):
    try:
          ok, frame = self.vs.read() cv2image =
    cv2.flip(frame, 1) hands = hd.findHands(cv2image,
    draw=False, flipType=True)
    cv2image_copy=np.array(cv2image) cv2image =
    cv2.cvtColor(cv2image, cv2.COLOR_BGR2RGB)
    self.current_image = Image.fromarray(cv2image)
```

```
imgtk = ImageTk.PhotoImage(image=self.current_image)
self.panel.imgtk = imgtk
self.panel.config(image=imgtk)
        if hands:
            # #print(" --------- lmlist=",hands[1])  hand = hands[0]
x, y, w, h = hand['bbox']  image = cv2image_copy[y - offset:y
+ h + offset, x - offset:x + w + offset]  white =
cv2.imread("C:/Users/Leena Ali/Documents/DataScienceProjects/Sign-
Language-To-Text-and-Speech-Conversion- master/white.jpg")
            # img_final=img_final1=img_final2=0
            handz = hd2.findHands(image, draw=False,
flipType=True)  print(" ", self.ccc)  self.ccc
+= 1  if handz:
                hand = handz[0]
self.pts = hand['lmList']  #
x1,y1,w1,h1=hand['bbox']
                os = ((400 - w) // 2) -
15  os1 = ((400 - h) //
2) - 15  for t in range(0,
4, 1):
                cv2.line(white, (self.pts[t][0] + os, self.pts[t][1] + os1),
(self.pts[t + 1][0] + os, self.pts[t + 1][1] + os1),
                         (0, 255, 0),
3)  for t in range(5,
8, 1):
                cv2.line(white, (self.pts[t][0] + os, self.pts[t][1] + os1),
(self.pts[t + 1][0] + os, self.pts[t + 1][1] + os1),
                         (0, 255, 0),
3)  for t in range(9,
12, 1):
                cv2.line(white, (self.pts[t][0] + os, self.pts[t][1] + os1),
(self.pts[t + 1][0] + os, self.pts[t + 1][1] + os1),
                         (0, 255, 0),
3)  for t in range(13,
16, 1):
```

```
                cv2.line(white, (self.pts[t][0] + os, self.pts[t][1] + os1),
    (self.pts[t + 1][0] + os, self.pts[t + 1][1] + os1),
                         (0, 255, 0),
    3)              for t in range(17,
    20, 1):
                cv2.line(white, (self.pts[t][0] + os, self.pts[t][1] + os1),
    (self.pts[t + 1][0] + os, self.pts[t + 1][1] + os1),
                         (0, 255, 0), 3)
 cv2.line(white, (self.pts[5][0] + os, self.pts[5][1] + os1), (self.pts[9][0] +
os,
     self.pts[9][1] + os1), (0, 255, 0),
                         3)
 cv2.line(white, (self.pts[9][0] + os, self.pts[9][1] + os1), (self.pts[13][0] +
     os, self.pts[13][1] + os1), (0, 255, 0),
                         3)
                cv2.line(white, (self.pts[13][0] + os, self.pts[13][1] + os1),
    (self.pts[17][0] + os, self.pts[17][1] + os1),
                         (0, 255, 0), 3)
 cv2.line(white, (self.pts[0][0] + os, self.pts[0][1] + os1), (self.pts[5][0] +
os,
     self.pts[5][1] + os1), (0, 255, 0),
                         3)
 cv2.line(white, (self.pts[0][0] + os, self.pts[0][1] + os1), (self.pts[17][0] +
     os, self.pts[17][1] + os1), (0, 255, 0),
                         3)
                for i in range(21):
 cv2.circle(white, (self.pts[i][0] + os, self.pts[i][1] + os1), 2, (0, 0, 255),
     1)
                res=white
    self.predict(res)
                self.current_image2 = Image.fromarray(res)
                imgtk = ImageTk.PhotoImage(image=self.current_image2)
                self.panel2.imgtk = imgtk
    self.panel2.config(image=imgtk)
```

```
                self.panel3.config(text=self.current_symbol,      font=("Courier",
    30))
                #self.panel4.config(text=self.word, font=("Courier", 30))
self.b1.config(text=self.word1,  font=("Courier",  20),  wraplength=825,
     command=self.action1)
self.b2.config(text=self.word2,  font=("Courier",  20),  wraplength=825,
     command=self.action2)
self.b3.config(text=self.word3,  font=("Courier",  20),  wraplength=825,
     command=self.action3)
self.b4.config(text=self.word4,  font=("Courier",  20),  wraplength=825,
     command=self.action4)
            self.panel5.config(text=self.str,           font=("Courier",           30),
    wraplength=1025)        except Exception:
            print("==",
    traceback.format_exc())         finally:
    self.root.after(1, self.video_loop)
       def distance(self,x,y):         return math.sqrt(((x[0] -
    y[0]) ** 2) + ((x[1] - y[1]) ** 2))
       def action1(self):
          idx_space      =      self.str.rfind("      ")
    idx_word        =        self.str.find(self.word,
    idx_space)              last_idx  =  len(self.str)
    self.str = self.str[:idx_word]          self.str =
    self.str + self.word1.upper()
       def action2(self):
          idx_space      =      self.str.rfind("      ")
    idx_word        =        self.str.find(self.word,
    idx_space)              last_idx  =  len(self.str)
    self.str=self.str[:idx_word]
    self.str=self.str+self.word2.upper()
    #self.str[idx_word:last_idx] = self.word2
       def action3(self):
          idx_space      =      self.str.rfind("      ")
    idx_word        =        self.str.find(self.word,
```

```
    idx_space)              last_idx = len(self.str)
    self.str = self.str[:idx_word]
          self.str = self.str + self.word3.upper()
       def action4(self):
          idx_space     =     self.str.rfind("     ")
    idx_word       =       self.str.find(self.word,
    idx_space)              last_idx = len(self.str)
    self.str = self.str[:idx_word]         self.str =
    self.str + self.word4.upper()
       def speak_fun(self):
          self.speak_engine.say(self.str)
    self.speak_engine.runAndWait()
       def   clear_fun(self):
    self.str="                  "
    self.word1    =    "    "
    self.word2    =    "    "
    self.word3    =    "    "
    self.word4 = " "
       def predict(self, test_image):
          white=test_image       white = white.reshape(1, 400,
    400, 3)       prob = np.array(self.model.predict(white)[0],
    dtype='float32')             ch1 = np.argmax(prob, axis=0)
    prob[ch1] = 0                ch2 = np.argmax(prob, axis=0)
    prob[ch2] = 0                ch3 = np.argmax(prob, axis=0)
    prob[ch3] = 0
          pl = [ch1, ch2]
          # condition for [Aemnst]
l = [[5, 2], [5, 3], [3, 5], [3, 6], [3, 0], [3, 2], [6, 4], [6, 1], [6, 2], [6,
6], [6, 7], [6, 0],
     [6, 5],
              [4, 1], [1, 0], [1, 1], [6, 3], [1, 6], [5, 6], [5, 1], [4, 5], [1, 4],
    [1, 5], [2, 0], [2, 6], [4, 6],
              [1, 0], [5, 7], [1, 6], [6, 1], [7, 6], [2, 5], [7, 1], [5, 4], [7, 0], [7,
    5], [7, 2]]         if pl in l:
if (self.pts[6][1] < self.pts[8][1] and self.pts[10][1] < self.pts[12][1] and
```

```
 self.pts[14][1] < self.pts[16][1] and self.pts[18][1] < self.pts[20][
                1]):
                ch1 = 0
                # print("00000")
        # condition for [o][s]        l =
    [[2, 2], [2, 1]]       if pl in l:          if
    (self.pts[5][0]    <    self.pts[4][0]):
    ch1 = 0

    print("++++++++++++++++++")
    # print("00000")
        # condition for [c0][aemnst]        l = [[0, 0], [0, 6], [0,
    2], [0, 5], [0, 1], [0, 7], [5, 2], [7, 6], [7, 1]]         pl = [ch1,
    ch2]        if pl in l:
if (self.pts[0][0] > self.pts[8][0] and self.pts[0][0] > self.pts[4][0] and
     self.pts[0][0] > self.pts[12][0] and self.pts[0][0] > self.pts[16][
                0] and self.pts[0][0] > self.pts[20][0]) and self.pts[5][0] >
    self.pts[4][0]:
                ch1 = 2
                # print("22222")
        # condition for [c0][aemnst]       l = [[6,
    0], [6, 6], [6, 2]]         pl = [ch1, ch2]         if
    pl in l:                if self.distance(self.pts[8],
    self.pts[16]) < 52:
                ch1 = 2
                # print("22222")
        #            condition            for
    [gh][bdfikruvw]        l = [[1, 4], [1,
    5], [1, 6], [1, 3], [1, 0]]          pl =
    [ch1, ch2]
        if pl in l:
if self.pts[6][1] >  self.pts[8][1] and self.pts[14][1] <  self.pts[16][1] and
     self.pts[18][1] < self.pts[20][1] and self.pts[0][0] < self.pts[8][
0]  and self.pts[0][0] <  self.pts[12][0] and self.pts[0][0] <  self.pts[16][0]
and
```

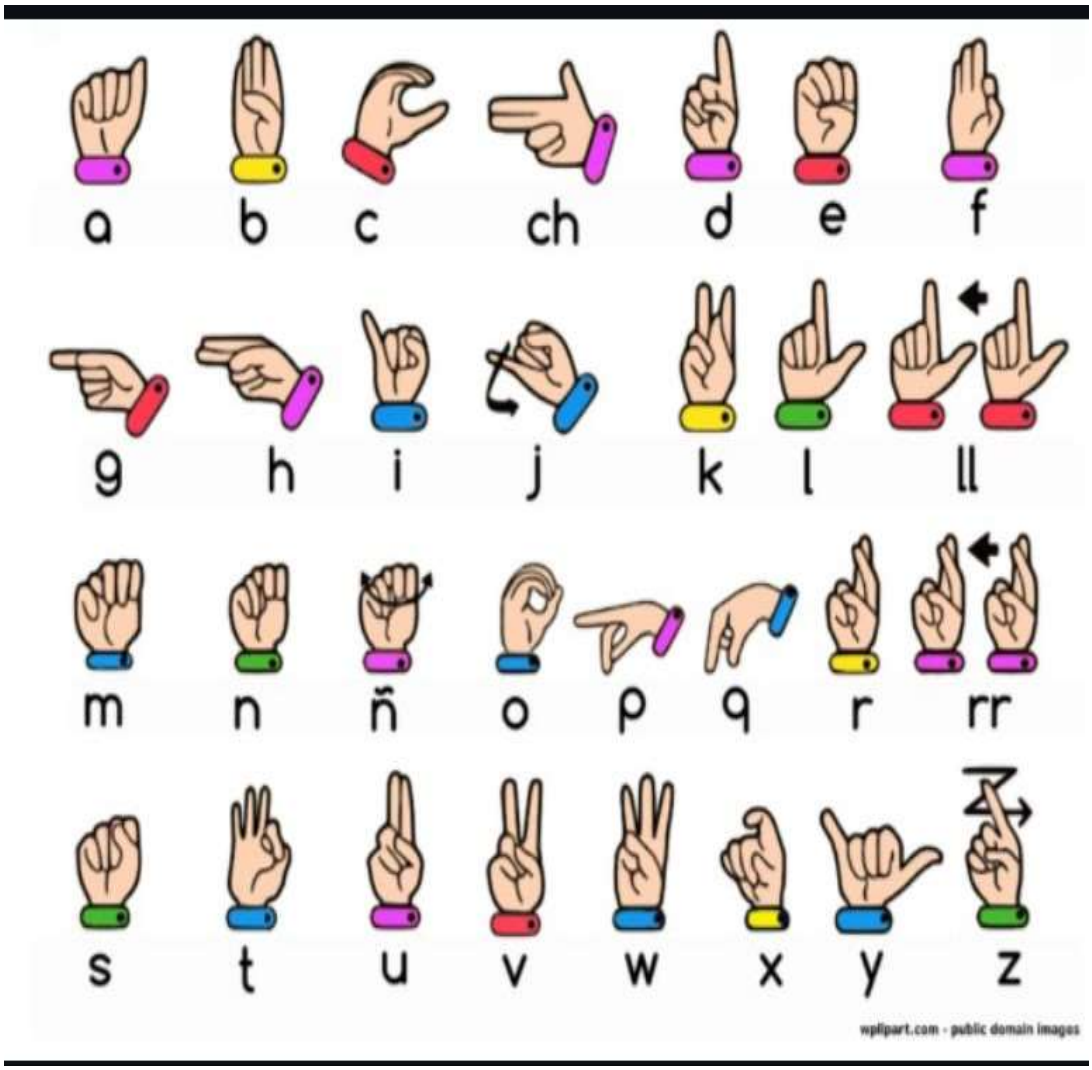
```
 self.pts[0][0] < self.pts[20][0]:
                ch1          =          3
    print("33333c")
          # con for [gh][l]       l = [[4, 6],
    [4, 1], [4, 5], [4, 3], [4, 7]]         pl =
    [ch1, ch2]          if pl in l:              if
    self.pts[4][0] > self.pts[0][0]:
                ch1          =          3
    print("33333b")
          # con for [gh][pqz]         l = [[5, 3], [5, 0],
    [5, 7], [5, 4], [5, 2], [5, 1], [5, 5]]         pl = [ch1,
    ch2]         if pl in l:              if self.pts[2][1] + 15
    < self.pts[16][1]:
                ch1          =          3
    print("33333a")
          # con for [l][x]        l = [[6, 4], [6, 1], [6,
    2]]        pl = [ch1, ch2]         if pl in l:           if
    self.distance(self.pts[4],  self.pts[11])  >  55:
    ch1 = 4
                # print("44444")
          # con for [l][d]       l
    =  [[1,  4],  [1,  6],  [1,  1]]
    pl = [ch1, ch2]         if pl
    in l:
              if (self.distance(self.pts[4], self.pts[11]) > 50) and (
self.pts[6][1] >   self.pts[8][1]  and  self.pts[10][1] <   self.pts[12][1]  and
     self.pts[14][1]     <      self.pts[16][1]      and
    self.pts[18][1] <                  self.pts[20][1]):
    ch1 = 4
                # print("44444")
          # con for [l][gh]         l = [[3,
    6], [3, 4]]        pl = [ch1, ch2]        if
    pl in l:               if (self.pts[4][0] <
    self.pts[0][0]):
                ch1 = 4
```

```
                # print("44444")
            # con for [l][c0]       l = [[2, 2],
        [2, 5], [2, 4]]           pl = [ch1, ch2]
        if pl in l:             if (self.pts[1][0] <
        self.pts[12][0]):
                ch1 = 4
                # print("44444")
            # con for [l][c0]       l = [[2, 2],
        [2, 5], [2, 4]]           pl = [ch1, ch2]
        if pl in l:             if (self.pts[1][0] <
        self.pts[12][0]):               ch1 = 4
                # print("44444")
            #  con  for  [gh][z]
        l = [[3, 6], [3, 5], [3, 4]]
        pl = [ch1, ch2]         if pl
        in l:
    if ( self.pts[6][1] >  self.pts[8][1]  and  self.pts[10][1] <  self.pts[12][1]  and
         self.pts[14][1]  <  self.pts[16][1]  and  self.pts[18][1]  <
        self.pts[20][             1]) and self.pts[4][1] > self.pts[10][1]:
                ch1          =          5
        print("55555b")
            #  con  for  [gh][pq]
        l = [[3, 2], [3, 1], [3, 6]]
        pl = [ch1, ch2]         if pl
        in l:
    if  self.pts[4][1]  +  17  >  self.pts[8][1]  and  self.pts[4][1]  +  17  >
    self.pts[12][1]  and
         self.pts[4][1]  +  17  >  self.pts[16][1]  and
        self.pts[4][              1] + 17 > self.pts[20][1]:
                ch1          =          5
        print("55555a")
            # con for [l][pqz]        l = [[4, 4], [4,
        5], [4, 2], [7, 5], [7, 6], [7, 0]]           pl =
        [ch1, ch2]            if pl in l:                 if
        self.pts[4][0] > self.pts[0][0]:
```

```
                ch1 = 5
                # print("55555")
        # con for [pqz][aemnst]         l = [[0, 2], [0, 6], [0, 1],
    [0, 5], [0, 0], [0, 7], [0, 4], [0, 3], [2, 7]]        pl = [ch1, ch2]
    if pl in l:
if self.pts[0][0] < self.pts[8][0] and self.pts[0][0] < self.pts[12][0] and
     self.pts[0][0] < self.pts[16][0] and self.pts[0][0] <
    self.pts[20][0]:                ch1 = 5
                # print("55555")
        # con for [pqz][yj]          l =
    [[5, 7], [5, 2], [5, 6]]           pl =
    [ch1, ch2]        if pl in l:           if
    self.pts[3][0] < self.pts[0][0]:
                ch1 = 7
                # print("77777")
```

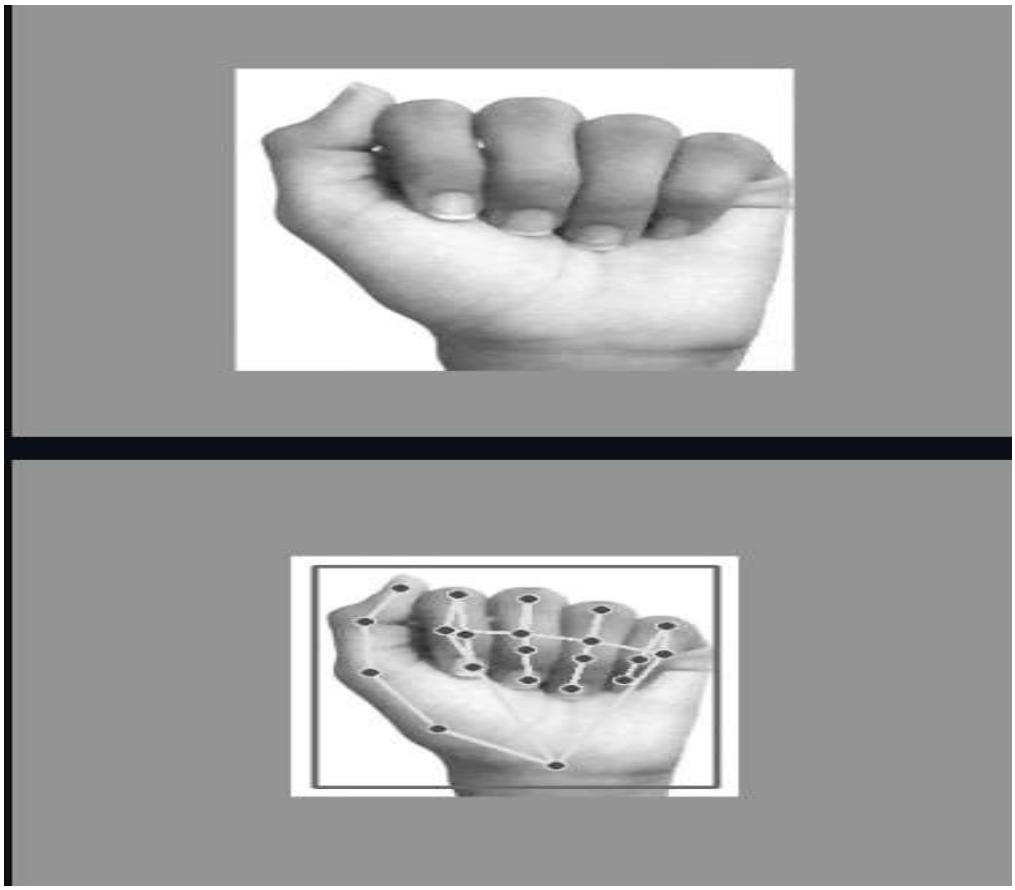
APPENDIX-2

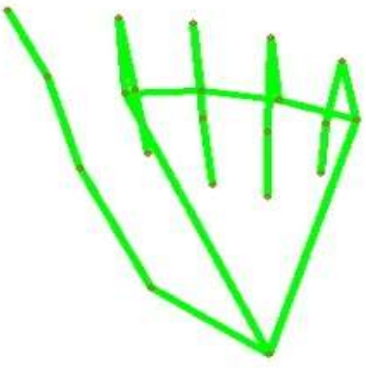
SNAPSHOT



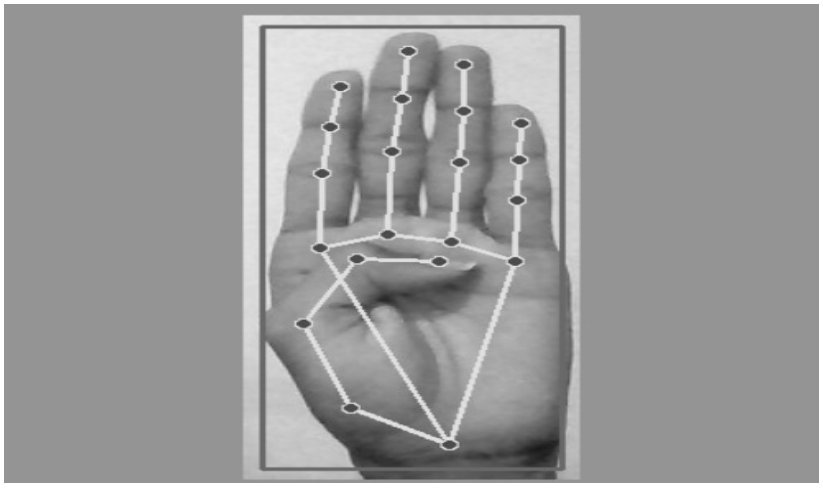
Representation Of Alphabets

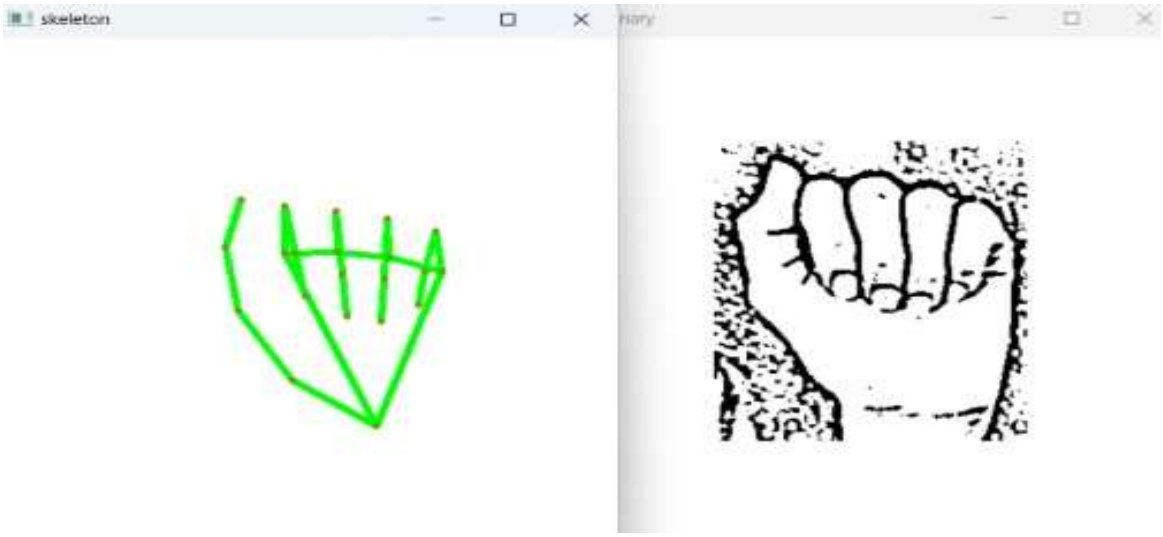
OUTPUT 1





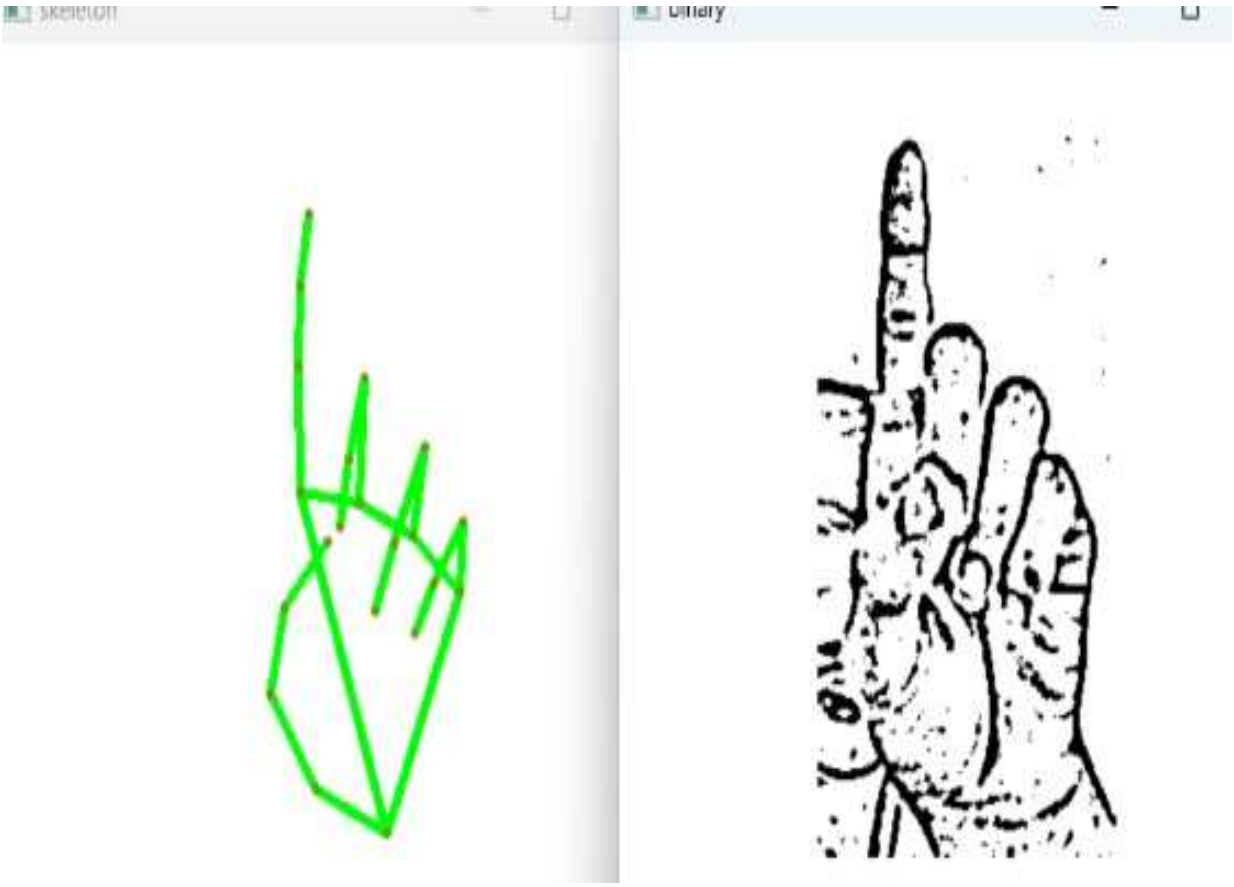
OUTPUT 2





OUTPUT 3

OUTPUT 4



OUTPUT 5

REFERENCES

1. **Rautaray, S. S., & Agrawal, A. (2015).**

Vision based hand gesture recognition for human computer interaction: a survey.

Artificial Intelligence Review, 43(1), 1-54.

https://doi.org/10.1007/s10462-012-9356-9

2. **Mitra, S., & Acharya, T. (2007).**

Gesture recognition: A survey. IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews, 37(3), 311-324.

https://doi.org/10.1109/TSMCC.2007.893280

3. **Kaur, H., & Arora, A. (2016).**

Sign language recognition system for deaf and dumb people. International

Journal of Engineering and Computer Science, 5(3).

https://www.ijecs.in/index.php/ijecs/article/view/2259

4. **Zafrulla, Z., Brashear, H., Starner, T., Hamilton, H., & Presti, P. (2011).**

American sign language recognition with the kinect. In Proceedings of the 13th international conference on multimodal interfaces (pp. 279-286). https://doi.org/10.1145/2070481.2070532

5. **Wachs, J. P., Kölsch, M., Stern, H., & Edan, Y. (2011).**

Vision-based hand-gesture applications.

Communications of the ACM, 54(2), 60-71.

https://doi.org/10.1145/1897816.1897838

6. **Kumar, P., & Reddy, M. M. (2021).**

Gesture recognition using machine learning and deep learning for sign language.

Procedia Computer Science, 192, 4279-4288.

https://doi.org/10.1016/j.procs.2021.09.200

7. **Kasemsap, K. (2018).**

Human-Computer Interaction and Modern Applications in Education. In "Handbook of Research on Mobile Devices and Smart Gadgets in K-12 Education," IGI Global.

https://doi.org/10.4018/978-1-5225-2706-0.ch012

8. **Chaudhary, A., Raheja, J. L., Das, K., & Raheja, S. (2011).**

Intelligent Approaches to interact with Machines using Hand Gesture Recognition in Natural way: A Survey.

International Journal of Computer Science & Engineering Survey, 2(1).

https://doi.org/10.5121/ijcses.2011.2101

9. **Yu, X., & Ebrahimi, T. (2016).**

An overview of visual hand gesture recognition with wearable sensors and video-based methods.

IEEE Transactions on Human-Machine Systems, 46(4), 498-509.

https://doi.org/10.1109/THMS.2016.2599204

10. **Molchanov, P., Gupta, S., Kim, K., & Kautz, J. (2015).**

Hand gesture recognition with 3D convolutional neural networks. In Proceedings of the IEEE conference on computer vision and pattern recognition workshops (pp. 1-7).

https://openaccess.thecvf.com/content_cvpr_workshops_2015/W15/html/Molchanov_Hand_Gesture_Recognition_2015_CVPR_paper.html

PROGRAM OUTCOMES

PO No.	Graduate Attribute	Program Outcomes (POs)
PO1	Engineering knowledge	Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization for the solution of complex engineering problems.
PO2	Problem analysis	Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	Design/development of solutions	Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for public health and safety, and cultural, societal, and environmental considerations.
PO4	Conduct investigations of complex Problems	Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	Modern tool usage	Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modeling to complex engineering activities, with an understanding of the limitations.
PO6	The engineer and society	Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO7	Environment and Sustainability	Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO8	Ethics	Apply ethical principles and common to professional ethics and responsibilities and norms of the engineering practice.

PO9	**Individual and team work**	Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10	**Communication**	Communicate effectively on complex engineering activities with the engineering community and with the society at large, such as, being able to comprehend and withe effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	**Project management and finance**	Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	**Life-long learning**	Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Mapping of Program outcomes with the Project titled **"IncludeEd- INTEGRATING SIGN LANGUAGE AND VISUAL AIDS"**

PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12

PROGRAM SPECIFIC OUTCOMES

B.E COMPUTER SCIENCE AND ENGINEERING

PSO No.	Program Specific Outcomes
PSO1	To analyze, design and develop computing solutions by applying foundational concepts of computer science and engineering.
PSO2	To apply software engineering principles and practices for developing quality software for scientific and business applications.
PSO3	To adapt to emerging Information and Communication Technologies (ICT) to innovate ideas and solutions to existing/novel problems

PSO1	PSO2	PSO3

Signature of Guide